

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Phát triển hệ thống phát hiện tấn công mạng bằng
phát hiện bất thường**

ĐỖ TUẤN MINH

minh.dt225741@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: PGS.TS. Nguyễn Linh Giang

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Phát triển hệ thống phát hiện tấn công mạng bằng
phát hiện bất thường

ĐỖ TUẤN MINH

minh.dt225741@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: PGS.TS. Nguyễn Linh Giang

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Em xin cảm ơn PGS.TS. Nguyễn Linh Giang đã trực tiếp hướng dẫn đồ án này — từ giai đoạn định hướng đề tài đến việc chỉ ra những vấn đề kỹ thuật còn thiếu sót trong báo cáo. Những phản hồi cụ thể của Thầy giúp em điều chỉnh đúng hướng thay vì tự tìm lỗi một mình.

Em cảm ơn các thầy cô Trường Công nghệ thông tin và Truyền thông, Đại học Bách Khoa Hà Nội đã xây dựng nền tảng kiến thức về mạng máy tính, học máy và hệ thống trong bốn năm qua — những kiến thức là tiền đề trực tiếp cho công việc này.

Cảm ơn gia đình và bạn bè đã kiên nhẫn trong giai đoạn đồ án căng thẳng nhất.

Và cảm ơn bản thân đã chăm chỉ, kiên trì với từng thực nghiệm dù không phải lần nào kết quả cũng như kỳ vọng.

Hà Nội, tháng 06 năm 2025

Sinh viên thực hiện

Đỗ Tuấn Minh

TÓM TẮT NỘI DUNG ĐỒ ÁN

Phát hiện xâm nhập mạng là bài toán phân loại lưu lượng mạng thành Benign và các loại tấn công trong thời gian thực. Hướng tiếp cận dựa trên dấu hiệu như Snort phát hiện hiệu quả các tấn công đã biết nhưng không phát hiện được biến thể mới và tấn công hành vi. Hướng tiếp cận dựa trên học máy cho phép học đặc trưng từ dữ liệu lịch sử, nhưng gặp hai thách thức chính trong triển khai thực tế: mất cân bằng lớp cực đoan (Benign chiếm trên 80% lưu lượng) và covariate shift khi mô hình huấn luyện trên dữ liệu lab được triển khai sang môi trường mạng khác.

Đồ án này phát triển hệ thống phát hiện xâm nhập mạng lai ghép kết hợp Snort và học máy, được xây dựng qua ba giai đoạn nghiên cứu tuần tự. Giai đoạn 1 thiết lập kiến trúc nền tảng trên bộ dữ liệu NSL-KDD: Autoencoder Anomaly Gate lọc lưu lượng bình thường ở tầng thứ nhất, Stacking Ensemble gồm FT-Transformer kết hợp LightGBM qua Meta Logistic Regression phân loại tấn công ở tầng thứ hai. Mô hình đạt Macro F1 = 0,6809 trên KDDTest+, mức cạnh tranh với đánh giá trung thực theo giao thức train/test tách biệt. Giai đoạn 2 tái thiết kế kiến trúc Two-Stage Cascade cho bộ dữ liệu CIC-IDS-2017: Gating Network phân loại 6 lớp (Benign, DoS, DDoS, PortScan, Brute Force, Suspicious) với ngưỡng 85% cho lớp Suspicious, Expert Network phân loại 5 lớp (Benign, Web Attack, Bot, Infiltration, Heartbleed) cho các flow Suspicious; hệ thống xuất tổng hợp 9 lớp. Asymmetric Ensemble Voting kết hợp FT-Transformer, Random Forest và KNN với luật bỏ phiếu bất đối xứng cho Botnet và Infiltration; Hard Negative Mining hai vòng cải thiện Botnet F1 từ 0,4787 lên 0,6512. Hệ thống đạt Accuracy 99,55% và Macro F1 = 0,9294 trên tập kiểm tra CIC-IDS-2017. Giai đoạn 3 giải quyết vấn đề covariate shift khi triển khai mô hình sang môi trường Testbed WSL2: mô hình CIC giảm từ 99,55% xuống 21,93% Accuracy khi áp dụng trực tiếp. Sau khi thu thập dữ liệu tấn công thực từ hai máy trên LAN và tái huấn luyện hoàn toàn, mô hình V8.5 đạt Macro F1 = 91,7% trên 5 lớp với BruteForce Recall = 90% trên tập kiểm định đa dạng miền.

Đồ án đóng góp: (1) kiến trúc Two-Stage Cascade với Asymmetric Ensemble Voting cho bài toán phân loại đa lớp mất cân bằng cực đoan; (2) Hard Negative Mining hai vòng cải thiện F1 lớp thiểu số vượt trội class weight đơn thuần; (3) quy trình chẩn đoán covariate shift định lượng và tái huấn luyện theo dữ liệu đích; (4) bằng chứng thực nghiệm cho thấy tập kiểm định đồng nhất miền có thể inflate metric lên 14% tuyệt đối so với tập kiểm định đa dạng miền.

Sinh viên thực hiện

Đỗ Tuấn Minh

ABSTRACT

Network intrusion detection requires classifying network traffic into benign and attack categories in real time. Signature-based systems such as Snort effectively detect known attacks but fail against previously unseen variants and behavioral exploits. Machine learning approaches can learn behavioral features from labeled data, yet face two practical challenges: severe class imbalance (benign traffic typically exceeds 80% of all flows) and covariate shift when a model trained on a benchmark dataset is deployed in a different network environment.

This thesis develops a hybrid network intrusion detection system combining Snort and machine learning, built through three sequential research phases. Phase 1 establishes a foundational two-stage architecture on NSL-KDD: an Autoencoder Anomaly Gate filters normal traffic at the first stage, while a Stacking Ensemble of FT-Transformer and LightGBM combined via a Meta Logistic Regression classifies attack types at the second stage. The model achieves Macro F1 = 0.6809 on KDDTest+ under a strict train/test separation protocol. Phase 2 redesigns the architecture as a Two-Stage Cascade for CIC-IDS-2017: a Gating Network performs 6-class classification (Benign, DoS, DDoS, PortScan, Brute Force, Suspicious) at an 85% Suspicious-confidence threshold, while an Expert Network classifies 5 output classes (Benign, Web Attack, Bot, Infiltration, Heartbleed) for flows forwarded from Stage 1; together they produce 9 distinct output labels. Asymmetric Ensemble Voting combines FT-Transformer, Random Forest (150 trees, class_weight=balanced), and KNN (K=16, uniform weighting) with asymmetric voting rules for Botnet and Infiltration; two rounds of Hard Negative Mining with $10\times$ sample replication improve Botnet F1 from 0.4787 to 0.6512. The system achieves 99.55% Accuracy and Macro F1 = 0.9294 on the CIC-IDS-2017 test set. Phase 3 addresses covariate shift when deploying the CIC model to a WSL2 testbed environment, where accuracy drops from 99.55% to 21.93% under direct transfer. After collecting real attack traffic from two machines on a home LAN and fully retraining, model V8.5 achieves Macro F1 = 91.7% across 5 classes, with BruteForce Recall = 90% on a domain-diverse validation set.

The main contributions are: (1) a Two-Stage Cascade architecture with Asymmetric Ensemble Voting for severely imbalanced multi-class intrusion detection; (2) two-round Hard Negative Mining that outperforms class-weight adjustment by up to 40.6% on minority classes; (3) a quantified covariate shift diagnosis and targeted retraining strategy; and (4) empirical evidence that domain-

homogeneous validation sets can inflate reported metrics by up to 14 percentage points compared to domain-diverse evaluation.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện tại và hạn chế	2
1.3 Mục tiêu và định hướng giải pháp	2
1.4 Đóng góp của đồ án	3
1.5 Bố cục đồ án	4
CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT	5
2.1 Ngữ cảnh bài toán phát hiện xâm nhập mạng.....	5
2.2 Các nghiên cứu liên quan	5
2.3 Kiến trúc Feature Tokenizer Transformer	6
2.3.1 Cơ chế mã hóa đặc trưng.....	6
2.3.2 Self-Attention và Transformer Block.....	7
2.3.3 Lợi thế so với MLP và mô hình cây trên dữ liệu dạng bảng	9
2.4 Các kỹ thuật xử lý mất cân bằng dữ liệu.....	9
2.4.1 Focal Loss	9
2.4.2 SMOTE	10
2.4.3 Weighted Random Sampling và Class Weights	11
2.5 Học chuyển giao và thích nghi miền	11
2.6 Hệ thống IDS lai ghép: phát hiện theo luật và theo hành vi bất thường.....	12
CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT.....	14
3.1 Tổng quan kiến trúc hệ thống.....	14
3.2 Giai đoạn 1: Xây dựng mô hình nền tảng trên NSL-KDD	15
3.2.1 Đặc điểm bộ dữ liệu NSL-KDD và quy trình tiền xử lý.....	15
3.2.2 Kiến trúc Two-Stage: Autoencoder và Stacking Ensemble.....	15

3.2.3 Chiến lược xử lý mất cân bằng	17
3.2.4 Giới hạn của NSL-KDD và động cơ chuyển sang CIC-IDS-2017....	18
3.3 Giai đoạn 2: Two-Stage Cascade NIDS trên CIC-IDS-2017	18
3.3.1 Bộ dữ liệu CIC-IDS-2017 và thách thức mất cân bằng cực đoan	18
3.3.2 Quy trình tiền xử lý và kỹ thuật hóa đặc trưng.....	19
3.3.3 Tầng 1: Gating Network.....	20
3.3.4 Tầng 2: Expert Network và Hard Negative Mining	21
3.3.5 Từ Expert Network đơn lẻ đến Asymmetric Ensemble Voting	21
3.3.6 Thiết kế Asymmetric Ensemble Voting	22
3.4 Giai đoạn 3: Thách nghiệm miền sang Testbed thực chiến	22
3.4.1 Chẩn đoán Covariate Shift	22
3.4.2 Tại sao Re-fit Scaler thất bại	23
3.4.3 Fine-tuning với Layer Freezing.....	23
3.4.4 Model Surgery: mở rộng không gian đặc trưng	24
3.5 Giai đoạn 4: Tái huấn luyện mô hình trên dữ liệu Testbed nội bộ	24
3.5.1 Đặc điểm bộ dữ liệu Testbed nội bộ	24
3.5.2 Bài toán PortScan: tại sao điều chỉnh thuật toán không đủ.....	25
3.5.3 Bài toán Brute Force: thiếu đa dạng pattern và giải pháp tăng cường dữ liệu	25
3.5.4 Vấn đề đồng nhất miền trong tập kiểm định và hệ quả lên độ tin cậy của metric.....	26
3.5.5 Mô hình cuối V8.5: thiết kế, huấn luyện và đánh giá	26
3.6 Triển khai Hybrid IDS End-to-End.....	27
3.6.1 Pipeline tích hợp Snort, CICFlowMeter và FT-Transformer.....	27
3.6.2 Hệ thống phát hiện theo phát lại	28
3.6.3 Môi trường giả lập tấn công xuyên thiết bị.....	29

CHƯƠNG 4. PHÂN TÍCH MÔ HÌNH.....	30
4.1 Phân tích Focal Loss trong điều kiện mất cân bằng cực đoan	30
4.2 Phân tích tính phân tách đặc trưng PortScan và Benign	31
4.3 Cơ sở lý thuyết của Hard Negative Mining	32
4.4 Lý thuyết Ensemble Voting và kiểm soát Variance	32
4.5 Cơ sở của Layer Freezing trong Transfer Learning	33
4.6 Giới hạn của dữ liệu flow-based tính: trường hợp Infiltration	34
CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM.....	36
5.1 Môi trường thực nghiệm, bộ dữ liệu và tham số đánh giá.....	36
5.1.1 Môi trường phần cứng và phần mềm	36
5.1.2 Bộ dữ liệu thực nghiệm.....	36
5.1.3 Các chỉ số đánh giá	37
5.2 Kết quả giai đoạn NSL-KDD	37
5.3 Kết quả Two-Stage Cascade NIDS trên CIC-IDS-2017.....	40
5.3.1 Hiệu quả kiến trúc Two-Stage so với mô hình 1-Stage.....	40
5.3.2 Đóng góp của Hard Negative Mining	40
5.3.3 Kết quả Ensemble Voting cuối cùng.....	41
5.4 Kết quả thích nghi miền	42
5.5 Kết quả mô hình trên Testbed nội bộ.....	44
5.5.1 Bằng chứng định lượng về tính không phân tách của PortScan và Benign	44
5.5.2 Kết quả phiên bản cuối	46
5.6 Thử nghiệm hệ thống End-to-End	47
5.6.1 Khả năng phát hiện theo loại tấn công.....	47
5.6.2 Hiệu năng định lượng trên corpus phát lại	49
5.6.3 Lỗi sai và bài học	49

KẾT LUẬN	51
Kết luận	51
Hướng phát triển trong tương lai.....	52
TÀI LIỆU THAM KHẢO.....	55
PHỤ LỤC.....	57
A. CẤU HÌNH MÔI TRƯỜNG THỰC NGHIỆM	57
A.1 Cấu hình WSL2 Mirrored Networking	57
A.2 Cài đặt và cấu hình Snort 3	57
A.3 Triển khai CICFlowMeter trên WSL.....	59
A.4 Hướng dẫn khởi chạy pipeline End-to-End	60
B. THÔNG SỐ MÔ HÌNH VÀ KẾT QUẢ CHI TIẾT	62
B.1 Siêu tham số mô hình	62
B.1.1 NSL-KDD — Stage 1: Autoencoder Anomaly Gate.....	62
B.1.2 NSL-KDD — Stage 2: FT-Transformer	62
B.1.3 NSL-KDD — Stage 2: LightGBM	63
B.1.4 NSL-KDD — Stage 2: Meta Logistic Regression	63
B.1.5 CIC-IDS-2017 — Stage 1: Gating Network	63
B.1.6 CIC-IDS-2017 — Stage 2: Expert Network.....	64
B.1.7 CIC-IDS-2017 — Stage 2: Random Forest và KNN.....	64
B.1.8 Testbed V8.5 — FT-Transformer.....	64
B.2 Kết quả chi tiết các thực nghiệm.....	65
B.2.1 NSL-KDD — Kết quả per-class trên KDDTest+.....	65
B.2.2 CIC-IDS-2017 — Kết quả ablation kiến trúc.....	65
B.2.3 CIC-IDS-2017 — Kết quả per-class cuối	66
B.2.4 Chẩn đoán covariate shift	66
B.2.5 Testbed V8.5 — Kết quả per-class trên val set đa dạng miền.....	66

B.2.6 So sánh val set đồng nhất và đa dạng miền.....	67
--	----

DANH MỤC HÌNH VẼ

Hình 2.1	Kiến trúc FT-Transformer: mỗi đặc trưng số thực được mã hóa thành token riêng, xử lý qua L tầng Self-Attention. Token [CLS] tổng hợp thông tin toàn chuỗi cho phân loại cuối.	8
Hình 3.1	Tổng quan quá trình nghiên cứu: từ khảo sát nền tảng đến triển khai End-to-End thực tế. Mỗi giai đoạn giải quyết một giới hạn phát sinh từ giai đoạn trước.	14
Hình 3.2	Phân phối lớp trong CIC-IDS-2017 (2,83 triệu flow, thang logarithm). Benign chiếm 82,7%; tỉ lệ Benign:Infiltration vượt 2.750:1.	19
Hình 3.3	Kiến trúc Two-Stage Cascade NIDS: Gating Network (6 lớp) lọc các tấn công phổ biến và chuyển Suspicious flows sang Expert Network (5 lớp). Hệ thống xuất 9 lớp tổng hợp từ 15 nhãn gốc. . . .	20
Hình 3.4	Kiến trúc mục tiêu của hệ thống Hybrid IDS End-to-End: Snort (tầng packet, rule-based) và FT-Transformer V8.5 (tầng flow, ML) chạy song song, alert được hợp nhất tại Alert Aggregator. Đường xử lý thời gian thực hoàn toàn là thiết kế mục tiêu; bản đã hiện thực và đánh giá là hệ thống phát hiện theo phát lại (Mục 3.6.2). 27	27
Hình 5.1	So sánh Macro F1 qua các phiên bản kiến trúc trên NSL-KDD (đánh giá đầu-cuối trên KDDTest+).	38
Hình 5.2	Kết quả phân loại đầu-cuối trên KDDTest+ — Stacking Ensemble (FTT + LightGBM → Meta-LR).	39
Hình 5.3	Cải thiện F1 của Botnet và Infiltration sau 2 vòng Hard Negative Mining (nhân bản 10×).	41
Hình 5.4	So sánh Expert Network (sau HNM) và Asymmetric Ensemble cuối cho hai lớp khó nhất.	42
Hình 5.5	So sánh Accuracy và MCC qua chuỗi can thiệp domain adaptation (CIC-IDS-2017 → Testbed WSL2).	44
Hình 5.6	PortScan F1 trên môi trường WSL/NAT với các thuật toán khác nhau — kết quả dưới 9% nhất quán trên mọi mô hình cho thấy vấn đề nằm ở dữ liệu, không phải thuật toán.	45
Hình 5.7	V8.5 — F1-score per-class trên tập validation đa dạng miền (11.183 flows). Đường ngang thể hiện Macro F1 = 0,917.	46

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các kiến trúc trên dữ liệu dạng bảng	9
Bảng 3.1	Thành phần tập dữ liệu huấn luyện V8.5	26
Bảng 4.1	F1-score PortScan qua các thử nghiệm thuật toán và siêu tham số khác nhau (V8.2)	31
Bảng 5.1	Tổng quan các bộ dữ liệu thực nghiệm	36
Bảng 5.2	So sánh Macro-F1 đầu-cuối (5 lớp, KDDTest+) qua các phiên bản kiến trúc	37
Bảng 5.3	Kết quả phân loại đầu-cuối trên KDDTest+ (Two-Stage: AE Gate + Stacking Ensemble)	38
Bảng 5.4	Kết quả Hard Negative Mining qua hai vòng trên hai lớp khó nhất	41
Bảng 5.5	So sánh Expert Network (sau HNM) và Asymmetric Ensemble cuối trên CIC-IDS-2017	42
Bảng 5.6	Kết quả qua chuỗi can thiệp domain adaptation	43
Bảng 5.7	PortScan F1 trên dữ liệu Testbed WSL với các thuật toán và cấu hình khác nhau	45
Bảng 5.8	Kết quả V8.5 trên tập validation đa dạng miền	46
Bảng 5.9	Kết quả phát hiện End-to-End theo kịch bản tấn công	48
Bảng 5.10	Tiến triển Macro F1 của hệ thống replay theo từng can thiệp hậu xử lý	49
Bảng B.1	Siêu tham số Autoencoder NSL-KDD Stage 1	62
Bảng B.2	Siêu tham số FT-Transformer NSL-KDD Stage 2	62
Bảng B.3	Siêu tham số LightGBM NSL-KDD Stage 2	63
Bảng B.4	Siêu tham số Meta Logistic Regression (Stacking)	63
Bảng B.5	Siêu tham số Gating Network FT-Transformer (CIC Stage 1)	63
Bảng B.6	Siêu tham số Expert Network FT-Transformer (CIC Stage 2)	64
Bảng B.7	Siêu tham số Random Forest và KNN (Ensemble)	64
Bảng B.8	Siêu tham số FT-Transformer Testbed V8.5	64
Bảng B.9	Kết quả per-class Stacking Ensemble trên KDDTest+	65
Bảng B.10	Ablation study các cải tiến kiến trúc trên CIC-IDS-2017	65
Bảng B.11	Kết quả per-class Asymmetric Ensemble Voting trên CIC- IDS-2017	66
Bảng B.12	Kết quả chuỗi can thiệp domain adaptation	66

Bảng B.13	Kết quả per-class mô hình V8.5 trên val set đa dạng miền . . .	66
Bảng B.14	Ảnh hưởng của tính đồng nhất miền trong tập kiểm định lên	
	BruteForce F1	67

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
AUC	Diện tích dưới đường cong ROC (Area Under the Curve)
CIC-IDS-2017	Bộ dữ liệu phát hiện xâm nhập của Viện An ninh mạng Canada năm 2017
DDoS	Tấn công từ chối dịch vụ phân tán (Distributed Denial of Service)
DL	Học sâu (Deep Learning)
DoS	Tấn công từ chối dịch vụ (Denial of Service)
EMA	Trung bình động hàm mũ (Exponential Moving Average)
FPR	Tỷ lệ báo động giả (False Positive Rate)
FT-Transformer	Kiến trúc Transformer cho dữ liệu dạng bảng (Feature Tokenizer Transformer)
IAT	Thời gian giữa hai gói tin liên tiếp (Inter-Arrival Time)
IDS	Hệ thống phát hiện xâm nhập (Intrusion Detection System)
KNN	K láng giềng gần nhất (K-Nearest Neighbors)
MCC	Hệ số tương quan Matthews (Matthews Correlation Coefficient)
ML	Học máy (Machine Learning)
MLP	Mạng nơ-ron nhiều lớp (Multilayer Perceptron)
NAT	Dịch địa chỉ mạng (Network Address Translation)
NIDS	Hệ thống phát hiện xâm nhập mạng (Network Intrusion Detection System)
NSL-KDD	Bộ dữ liệu chuẩn phát hiện xâm nhập của Phòng thí nghiệm An ninh mạng, dựa trên KDD Cup 1999

Thuật ngữ	Ý nghĩa
RF	Rừng ngẫu nhiên (Random Forest)
SMOTE	Kỹ thuật tổng hợp dữ liệu cho lớp thiểu số (Synthetic Minority Over-sampling Technique)
SOC	Trung tâm vận hành an ninh mạng (Security Operations Center)
WSL	Hệ thống Linux trên Windows (Windows Subsystem for Linux)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong thập kỷ vừa qua, các mối đe dọa mạng ngày càng tinh vi và có tổ chức, đặt ra thách thức lớn cho hạ tầng bảo mật của tổ chức ở mọi quy mô. Theo báo cáo Cost of a Data Breach 2023 của IBM [1], chi phí trung bình của một vụ vi phạm dữ liệu đã đạt 4,45 triệu USD — mức cao nhất trong 18 năm lịch sử báo cáo. Thời gian phát hiện trung bình vẫn kéo dài tới 204 ngày, nghĩa là kẻ tấn công có thể hoạt động bên trong hệ thống gần 7 tháng trước khi bị phát hiện. Tại Việt Nam, Cục An toàn thông tin ghi nhận hàng chục nghìn sự cố an ninh mạng mỗi năm, với mục tiêu ngày càng đa dạng từ cơ quan chính phủ đến doanh nghiệp vừa và nhỏ.

Hệ thống phát hiện xâm nhập mạng (NIDS) là cơ chế giám sát chủ động quan trọng trong kiến trúc phòng thủ đa tầng. Thay vì chỉ ngăn chặn thụ động tại tường lửa, NIDS phân tích liên tục luồng lưu lượng mạng và cảnh báo khi phát hiện hành vi bất thường, cho phép đội ngũ vận hành an ninh (SOC) phản ứng sớm trước khi thiệt hại lan rộng. Tuy nhiên, bài toán phát hiện xâm nhập trong thực tế phức tạp hơn nhiều so với các benchmark nghiên cứu. Ba thách thức cốt lõi mà đề tài này trực tiếp đối mặt là:

- **Mất cân bằng dữ liệu cực đoan:** Trong môi trường production, lưu lượng Benign thường chiếm trên 80% tổng số flow. Các lớp tấn công hiếm như Infiltration hay Botnet có thể chiếm dưới 0,1%. Một mô hình chỉ dự đoán tất cả là Benign đã đạt accuracy trên 80% nhưng hoàn toàn vô dụng về mặt bảo mật — bài toán này không thể đánh giá bằng accuracy thông thường.
- **Covariate Shift giữa dataset nghiên cứu và môi trường triển khai:** Các bộ dữ liệu nghiên cứu chuẩn được thu thập trong điều kiện lab kiểm soát — phần cứng chuyên dụng, switch vật lý, kịch bản tấn công cố định. Khi mô hình được triển khai sang môi trường thực tế với hạ tầng khác (NAT, card mạng ảo, router thông thường), phân phối đặc trưng thay đổi và hiệu năng sụt giảm nghiêm trọng. Đây là khoảng cách giữa nghiên cứu và thực chiến mà phần lớn tài liệu hiện tại bỏ qua.
- **Giới hạn của IDS dựa trên dấu hiệu:** Snort và các IDS signature-based truyền thống chỉ phát hiện được những tấn công đã được mã hóa thành luật cụ thể. Chúng không có khả năng phát hiện biến thể mới, công cụ tấn công hiện đại, hay kỹ thuật zero-day — chính xác là những gì kẻ tấn công có động cơ cao sẽ sử dụng.

1.2 Các giải pháp hiện tại và hạn chế

IDS dựa trên dấu hiệu là hướng tiếp cận phổ biến nhất trong sản xuất thực tế. Snort [2], IDS mã nguồn mở được triển khai rộng rãi nhất thế giới, duy trì cơ sở dữ liệu hàng chục nghìn luật mô tả pattern của các tấn công đã biết, xử lý traffic ở tốc độ gigabit với tỷ lệ false positive thấp. Hạn chế căn bản chỉ có một nhưng nghiêm trọng: bất kỳ tấn công nào chưa có luật — dù là biến thể nhỏ của tấn công đã biết — đều bị bỏ qua hoàn toàn. Trong bối cảnh các chiến dịch tấn công ngày càng sử dụng công cụ tùy biến và kỹ thuật evasion, giới hạn này ngày càng rõ nét.

IDS dựa trên bất thường thống kê xây dựng profile hành vi bình thường và cảnh báo khi có độ lệch đáng kể. Hướng này về lý thuyết có thể phát hiện tấn công zero-day, nhưng tỷ lệ false alarm cao là trở ngại thực tế lớn. Một buổi backup đêm, một cơn tăng đột biến video streaming, hay một bản cập nhật phần mềm lớn đều có thể kích hoạt cảnh báo. Tình trạng *alert fatigue* — đội ngũ SOC bị choáng ngợp bởi hàng trăm cảnh báo giả mỗi ngày đến mức bắt đầu bỏ qua cả cảnh báo thật — là vấn đề thực tiễn nghiêm trọng trong nhiều tổ chức.

IDS dựa trên học máy học đặc trưng hành vi từ dữ liệu lịch sử có nhãn, thay vì viết thủ công từng luật hay định nghĩa ngưỡng thống kê. Các nghiên cứu gần đây trên CIC-IDS-2017 [3] đạt Macro F1 trên 0,95 với Random Forest hay XGBoost. Tuy nhiên, phần lớn các kết quả này được đo trên chính dataset bằng cross-validation — và sụt giảm đáng kể khi triển khai sang môi trường mạng khác. Thực tế của đề tài xác nhận điều đó: mô hình FT-Transformer đạt Accuracy 99,55% trên tập kiểm tra CIC chỉ đạt 21,93% Accuracy khi áp dụng trực tiếp lên traffic thu thập từ môi trường WSL Testbed — một sụt giảm cho thấy covariate shift là vấn đề thực tế, không phải lý thuyết.

Kiến trúc học sâu cho dữ liệu dạng bảng — nhóm đặc trưng mà dữ liệu lưu lượng mạng thuộc về — từ lâu bị coi là địa hạt của các mô hình cây (Random Forest, XGBoost). Các mạng nơ-ron truyền thống (MLP) thường thua kém mô hình cây trên dữ liệu tabular vì thiếu cơ chế nắm bắt tương tác đặc trưng hiệu quả [4]. Feature Tokenizer Transformer (FT-Transformer) — kiến trúc áp dụng cơ chế Attention của Transformer lên từng đặc trưng riêng biệt — đã thu hẹp khoảng cách này và là nền tảng mô hình của đề tài.

1.3 Mục tiêu và định hướng giải pháp

Đề tài này không nhắm đến việc phủ nhận các hướng tiếp cận hiện tại, mà xây dựng một hệ thống NIDS lai ghép giải quyết đồng thời ba hạn chế nêu trên trong điều kiện phần cứng thực tế — một máy tính cá nhân chạy Windows với WSL, không có switch chuyên dụng hay thiết bị mạng đặc biệt.

Định hướng giải pháp được triển khai qua ba giai đoạn nghiên cứu có trình tự nhân-quả rõ ràng, mỗi giai đoạn được thúc đẩy bởi kết quả định lượng của giai đoạn trước:

Giai đoạn 1 thiết lập mô hình nền tảng trên NSL-KDD [5], xây dựng kiến trúc Two-Stage (Autoencoder Anomaly Gate + Stacking Ensemble FTT+LightGBM), pipeline tiền xử lý và các kỹ thuật xử lý mất cân bằng. Giai đoạn này xác định giới hạn của NSL-KDD — bộ dữ liệu từ năm 1998, không chứa các loại tấn công hiện đại — qua R2L Recall = 0,2523 do protocol shift không thể vượt được bằng thuật toán. Kết quả: Macro F1 = 0,6809.

Giai đoạn 2 tái thiết kế toàn bộ trên CIC-IDS-2017 [3] với 77 đặc trưng flow. Kiến trúc Two-Stage Cascade: Gating Network 6-lớp (Benign, DoS, DDoS, PortScan, Brute Force, Suspicious) + Expert Network 5-lớp (Benign, Web Attack, Bot, Infiltration, Heartbleed) xuất 9 lớp tổng hợp từ 15 nhãn gốc. Hard Negative Mining 2 vòng và Asymmetric Ensemble Voting với FT-Transformer, Random Forest và KNN được phát triển. Kết quả: Accuracy 99,55%, Macro F1 0,9294.

Giai đoạn 3 giải quyết covariate shift: mô hình CIC chỉ đạt 21,93% Accuracy khi áp dụng trực tiếp lên Testbed WSL (MCC = -0,015), trong khi Re-fit Scaler đơn thuần còn làm tình trạng xấu hơn (6,87%). Sau chẩn đoán định lượng, giải pháp là thu thập dữ liệu Testbed đặc thù và tái huấn luyện hoàn toàn. Mô hình V8.5 giải quyết hai bài toán đặc thù WSL: PortScan không phân tách trong NAT (dữ liệu surrogate từ CIC Friday hardware) và BruteForce thiếu diversity (mixed hydra + CIC Patator). Kết quả: Macro F1 91,7% trên val set đa dạng miền.

Hai tầng được tích hợp thành một hệ thống lai ghép: Snort ở tầng gói (rule-based) bù đắp cho FT-Transformer ở tầng flow (ML-based) để đạt độ bao phủ tấn công rộng hơn bất kỳ hướng tiếp cận đơn lẻ nào. Hệ thống được hiện thực hóa và đánh giá ở dạng phát hiện theo phát lại (replay-based live detection); đường xử lý thời gian thực hoàn toàn được để lại như một hướng phát triển.

1.4 Đóng góp của đồ án

Đồ án có bốn đóng góp chính:

1. **Kiến trúc Two-Stage Cascade NIDS với Asymmetric Ensemble Voting:**
Thiết kế hệ thống phân tầng Gating Network 6-lớp ($d = 128$, 4 lớp Attention, ngưỡng 85% cho lớp Suspicious) + Expert Network 5-lớp, kết hợp bỏ phiếu bất đối xứng giữa FT-Transformer, Random Forest (150 cây, max_depth=25, class_weight=balanced) và KNN (K=16). Đạt Accuracy 99,55% và Macro F1 0,9294 trên CIC-IDS-2017 (9 lớp tổng hợp từ 15 nhãn gốc).

2. **Hard Negative Mining 2 vòng cho lớp tấn công hiểm:** Thu thập mẫu bị phân loại sai sau mỗi vòng huấn luyện và nhân bản 10 lần bổ sung vào tập huấn luyện tiếp theo. Botnet F1 tăng 36% (0,4787→0,6512) và Infiltration F1 tăng 54,5% (0,3821→0,5903) — vượt trội class weight đơn thuần với +24,8% và +40,6% tương ứng.
3. **Chẩn đoán covariate shift định lượng và xây dựng Testbed thực:** Định lượng mức độ covariate shift: direct transfer Accuracy 21,93% (MCC −0,015); Re-fit Scaler làm tình trạng xấu hơn (6,87%). Thu thập dataset V8.5 (111.825 flows, 5 lớp) trong môi trường mạng LAN thực; xác định và giải quyết PortScan không phân tách trong NAT bằng dữ liệu surrogate; đánh giá robustness với val set đa dạng miền. Mô hình V8.5 đạt Macro F1 91,7%.
4. **Hệ thống Hybrid IDS phát hiện theo phát lại trên phần cứng phổ thông:** Hệ thống tích hợp Snort 3 và FT-Transformer V8.5 trên WSL2 Ubuntu, không yêu cầu GPU, được hiện thực hóa ở dạng phát hiện theo phát lại với giao diện thời gian thực. Thực nghiệm xác nhận tính bổ sung hai tầng: DoS slowhttptest chỉ phát hiện được bởi ML, XSS payload ngắn chỉ phát hiện được bởi Snort. Một luật hậu xử lý kiểu dấu hiệu (đếm cổng đích theo nguồn) bù cho đặc trưng quét cổng mà mô hình thiếu, nâng PortScan F1 từ 0,000 lên 0,996 mà không cần huấn luyện lại. Đường xử lý thời gian thực hoàn toàn được đề xuất như một hướng phát triển.

1.5 Bố cục đồ án

Đồ án được tổ chức thành sáu chương. Chương 2 trình bày nền tảng lý thuyết — kiến trúc FT-Transformer, các kỹ thuật xử lý mất cân bằng (Focal Loss, SMOTE, Weighted Sampling), học chuyển giao và khái niệm IDS lai ghép. Chương 3 mô tả chi tiết phương pháp đề xuất qua ba giai đoạn, từ mô hình NSL-KDD nền tảng đến pipeline Hybrid IDS End-to-End. Chương 4 phân tích lý thuyết các lựa chọn kỹ thuật quan trọng: tại sao Focal Loss hiệu quả hơn Cross-Entropy trong điều kiện mất cân bằng cực đoan, tại sao PortScan không thể phân tách trong không gian đặc trưng WSL, và giới hạn căn bản của dữ liệu flow-based với tấn công Infiltration. Chương 5 trình bày kết quả thực nghiệm đầy đủ qua từng giai đoạn, bao gồm phân tích ablation và kết quả hệ thống End-to-End. Chương 6 tổng kết các kết quả đạt được và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

2.1 Ngưỡng bài toán phát hiện xâm nhập mạng

Phát hiện xâm nhập mạng có thể được hình thức hóa như một bài toán phân loại có giám sát trên dữ liệu lưu lượng mạng. Đầu vào là tập hợp các đặc trưng thống kê mô tả một *flow* — chuỗi các gói tin giữa hai đầu cuối xác định bởi bộ 5-tuple (IP nguồn, IP đích, cổng nguồn, cổng đích, giao thức) trong một khoảng thời gian nhất định. Đầu ra là nhãn lớp: Benign (lưu lượng bình thường) hoặc một trong các loại tấn công đang xét.

Phương pháp trích xuất đặc trưng phổ biến nhất trong nghiên cứu IDS hiện đại là phân tích ở tầng flow, thay vì tầng gói tin (packet-level). Công cụ CICFlowMeter [6] trích xuất từ mỗi flow một vector gồm 77–80 đặc trưng thống kê, bao gồm: thời lượng flow, số lượng gói tin theo cả hai hướng (forward/backward), các thống kê về độ dài gói tin (trung bình, độ lệch chuẩn, tối thiểu, tối đa), thời gian giữa các gói tin liên tiếp (IAT), số lượng flag TCP, và các chỉ số tốc độ (bytes/giây, gói tin/giây). Cách tiếp cận này có ưu điểm là không cần giải mã payload — hoạt động được cả với lưu lượng mã hóa — nhưng cũng có giới hạn căn bản trong việc phát hiện các tấn công ẩn hoàn toàn trong nội dung gói tin (sẽ được phân tích chi tiết ở Mục 4.6).

Bài toán phân loại flow-based IDS phải đối mặt với thách thức mất cân bằng dữ liệu không điển hình. Ngay trong nhóm tấn công, sự phân tầng kích thước cũng cực đoan: DoS/DDoS có thể có hàng triệu flow trong khi Infiltration chỉ có vài nghìn. Điều này đòi hỏi metric phù hợp hơn là accuracy — cụ thể là Macro F1 (đánh giá đều tất cả các lớp không kể kích thước) và Matthews Correlation Coefficient (MCC) cho đánh giá tổng thể trên dữ liệu mất cân bằng cực đoan.

2.2 Các nghiên cứu liên quan

Lĩnh vực IDS dựa trên học máy đã trải qua nhiều thế hệ phát triển. Giai đoạn đầu (2000–2015) chủ yếu sử dụng Decision Tree, Naive Bayes, SVM trên bộ dữ liệu KDD Cup 1999 [7] và phiên bản cải tiến NSL-KDD [5]. NSL-KDD giải quyết một số vấn đề của KDD Cup gốc (loại bỏ bản ghi trùng lặp, cân bằng lại phân phối lớp) nhưng vẫn là dữ liệu từ năm 1998 — không phản ánh các giao thức và kỹ thuật tấn công hiện đại.

Sự ra đời của CIC-IDS-2017 [3] đánh dấu bước tiến quan trọng: dữ liệu thu thập từ mạng thực tế vào năm 2017 với 15 nhãn gốc (Benign và 14 loại tấn công hiện đại), được gộp thành 9 lớp mô hình, sử dụng CICFlowMeter để trích xuất 77 đặc trưng flow có ngữ nghĩa rõ ràng. Nhiều nghiên cứu trên CIC-IDS-2017 đạt

Accuracy trên 99% và F1 trên 0,95 với Random Forest và XGBoost [8], [9]. Tuy nhiên, phần lớn các kết quả này được đánh giá trong điều kiện in-distribution và ít quan tâm đến khả năng triển khai thực tế trên hạ tầng khác.

Về kiến trúc học sâu cho dữ liệu bảng, Gorishniy và cộng sự [4] đề xuất FT-Transformer — áp dụng kiến trúc Transformer lên từng đặc trưng riêng biệt. Trên nhiều benchmark tabular, FT-Transformer đạt kết quả cạnh tranh với XGBoost, đặc biệt trên các tập dữ liệu có nhiều đặc trưng liên tục. Với dữ liệu IDS có 77 đặc trưng số thực, FT-Transformer là lựa chọn phù hợp hơn MLP truyền thống nhờ cơ chế Attention học tương tác đặc trưng tường minh.

Về học chuyển giao trong IDS, hầu hết công trình hiện có [10], [11] tập trung vào chuyển giao giữa các dataset cùng không gian đặc trưng. Vấn đề covariate shift do sự khác biệt hạ tầng thu thập dữ liệu (switch vật lý vs. môi trường ảo hóa) là khoảng trống nghiên cứu mà đề tài này trực tiếp giải quyết. Kỹ thuật Layer Freezing và Model Surgery được đề xuất trong đề tài không có tiền lệ trực tiếp trong tài liệu IDS đã khảo sát.

2.3 Kiến trúc Feature Tokenizer Transformer

FT-Transformer [4] là kiến trúc Transformer được thiết kế riêng cho dữ liệu dạng bảng. Ý tưởng cốt lõi là xử lý mỗi *đặc trưng* như một *token* — tương tự cách Transformer trong NLP xử lý mỗi từ như một token — qua đó cho phép cơ chế Attention học tương tác giữa các đặc trưng một cách tường minh.

2.3.1 Cơ chế mã hóa đặc trưng

Cho một mẫu đầu vào $\mathbf{x} = [x_1, x_2, \dots, x_k] \in \mathbb{R}^k$ với k đặc trưng số thực (trường hợp của dữ liệu CICFlowMeter), bộ mã hóa đặc trưng (Feature Tokenizer) ánh xạ mỗi đặc trưng scalar x_j thành một vector nhúng $\mathbf{e}_j \in \mathbb{R}^d$:

$$\mathbf{e}_j = x_j \cdot \mathbf{w}_j + \mathbf{b}_j, \quad j = 1, 2, \dots, k \quad (2.1)$$

trong đó $\mathbf{w}_j \in \mathbb{R}^d$ là vector trọng số và $\mathbf{b}_j \in \mathbb{R}^d$ là vector bias, cả hai *đặc trưng riêng* cho từng đặc trưng j . Đây là phép chiếu tuyến tính theo từng đặc trưng: mỗi đặc trưng có tập tham số riêng, không dùng chung như trong MLP thông thường. Số tham số của Feature Tokenizer là $k \times 2d$.

Ngoài k vector đặc trưng, một token đặc biệt $\mathbf{e}_{[\text{CLS}]} \in \mathbb{R}^d$ — được khởi tạo ngẫu nhiên và học cùng với mô hình — được thêm vào đầu chuỗi. Token này đóng vai trò là đại diện tổng hợp toàn bộ input sau khi đi qua các tầng Attention và được dùng cho dự đoán cuối cùng. Ma trận token đầy đủ là:

$$\mathbf{T} = \begin{bmatrix} \mathbf{e}_{[\text{CLS}]} \\ \mathbf{e}_1 \\ \vdots \\ \mathbf{e}_k \end{bmatrix} \in \mathbb{R}^{(k+1) \times d} \quad (2.2)$$

Chiều d được gọi là chiều token (token dimension) và là siêu tham số quan trọng — trong đề tài này $d = 128$.

2.3.2 Self-Attention và Transformer Block

Mỗi tầng Transformer bao gồm hai thành phần chính: Multi-Head Self-Attention (MHSA) và Feed-Forward Network (FFN), mỗi thành phần được bao bởi kết nối tắt (residual connection) và chuẩn hóa lớp (Layer Normalization).

Multi-Head Self-Attention cho phép mỗi token chú ý đến tất cả các token khác trong chuỗi, học được mức độ ảnh hưởng giữa các đặc trưng. Với đầu vào $\mathbf{T}$, cơ chế Attention tính ba ma trận Query, Key, Value:

$$\mathbf{Q} = \mathbf{T}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{T}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{T}\mathbf{W}^V \quad (2.3)$$

trong đó $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d \times d_k}$ là các ma trận chiếu học được. Điểm Attention giữa hai vị trí phản ánh mức độ liên quan, sau đó được chuẩn hóa và dùng để tổng hợp Value:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (2.4)$$

Hệ số $1/\sqrt{d_k}$ chuẩn hóa tích vô hướng để tránh gradient vanishing khi d_k lớn. MHSA chạy h đầu Attention song song, mỗi đầu học một dạng tương tác khác nhau, rồi ghép kết quả:

$$\text{MHSA}(\mathbf{T}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (2.5)$$

$$\text{head}_i = \text{Attention}(\mathbf{T}\mathbf{W}_i^Q, \mathbf{T}\mathbf{W}_i^K, \mathbf{T}\mathbf{W}_i^V) \quad (2.6)$$

Feed-Forward Network là mạng hai lớp với hàm kích hoạt GELU, áp dụng độc lập lên từng vị trí token:

$$\text{FFN}(\mathbf{z}) = \text{GELU}(\mathbf{z}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2 \quad (2.7)$$

trong đó $W_1 \in \mathbb{R}^{d \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d}$, và $d_{ff} = 4d$ theo thông lệ. Một **Transformer Block** hoàn chỉnh kết hợp hai thành phần trên với residual và LayerNorm:

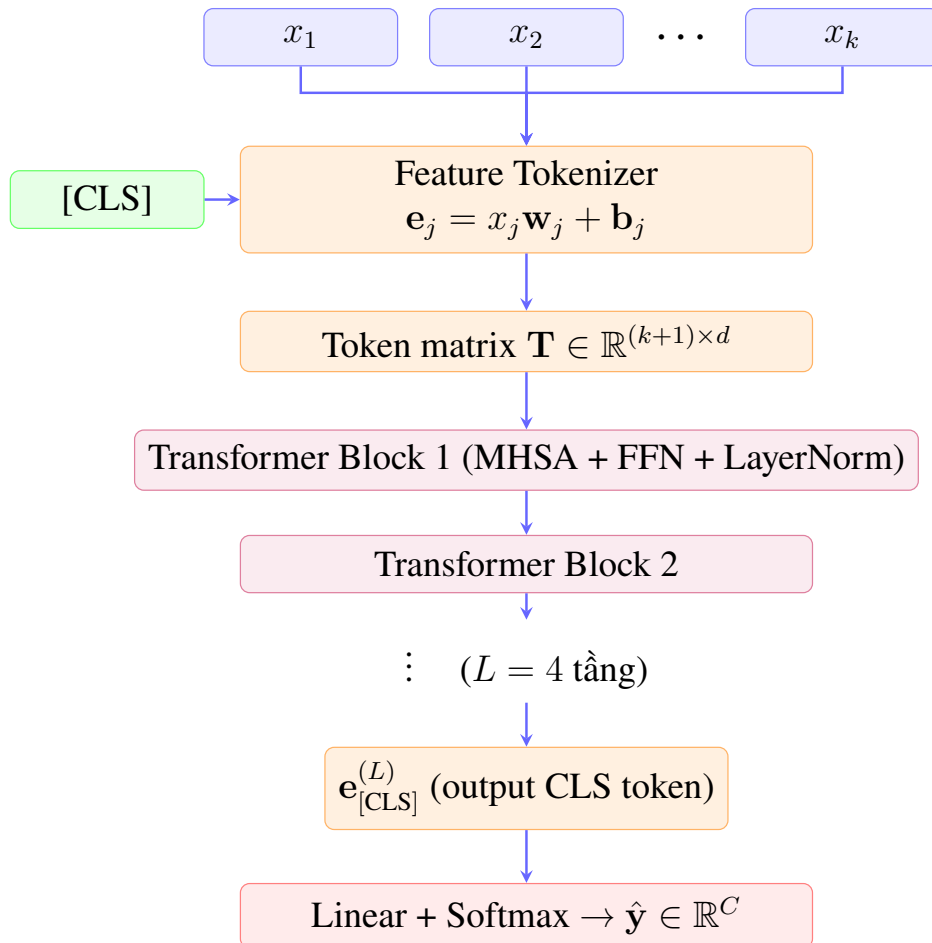
$$\mathbf{T}' = \text{LayerNorm}(\mathbf{T} + \text{MHSA}(\mathbf{T})) \quad (2.8)$$

$$\mathbf{T}'' = \text{LayerNorm}(\mathbf{T}' + \text{FFN}(\mathbf{T}')) \quad (2.9)$$

Sau L tầng Transformer, vector của token $[\text{CLS}]$ — ký hiệu $\mathbf{e}_{[\text{CLS}]}^{(L)}$ — đi qua một lớp tuyến tính và softmax để tạo phân phối xác suất trên C lớp:

$$\hat{\mathbf{y}} = \text{softmax}\left(\mathbf{e}_{[\text{CLS}]}^{(L)} W_{cls} + \mathbf{b}_{cls}\right) \quad (2.10)$$

Cấu hình FT-Transformer sử dụng trong đề tài cho Giai đoạn 2 (CIC-IDS-2017): $d = 128$, $h = 8$ đầu Attention, $L = 4$ tầng, $d_{ff} = 512$.



Hình 2.1: Kiến trúc FT-Transformer: mỗi đặc trưng số thực được mã hóa thành token riêng, xử lý qua L tầng Self-Attention. Token $[\text{CLS}]$ tổng hợp thông tin toàn chuỗi cho phân loại cuối.

2.3.3 Lợi thế so với MLP và mô hình cây trên dữ liệu dạng bảng

Điểm khác biệt căn bản giữa FT-Transformer và MLP thông thường nằm ở cách xử lý đặc trưng. Trong MLP, tất cả đặc trưng được nới thành một vector và xử lý qua các lớp Dense dùng chung trọng số — các đặc trưng tương tác với nhau một cách ẩn và không có cơ chế rõ ràng để mô hình tập trung vào cặp đặc trưng nào quan trọng hơn. FT-Transformer mã hóa mỗi đặc trưng thành một token riêng biệt, cho phép cơ chế Attention học tương minh *đặc trưng nào nên ảnh hưởng đến đặc trưng nào* — tương tự cách Random Forest học tương tác đặc trưng qua các điểm phân nhánh, nhưng linh hoạt hơn do không bị giới hạn bởi cấu trúc cây.

So với các mô hình cây (Random Forest, XGBoost), FT-Transformer có khả năng tổng quát hóa tốt hơn khi có nhiều đặc trưng liên tục với phân phối lệch — chính xác là đặc điểm của dữ liệu CICFlowMeter, nơi nhiều đặc trưng (IAT, packet length, byte count) có phân phối power-law hoặc log-normal. Ngoài ra, là mạng nơ-ron, FT-Transformer hỗ trợ fine-tuning tham số có chọn lọc (Layer Freezing) trong giai đoạn thích nghi miền, điều mà mô hình cây không hỗ trợ.

Bảng 2.1: So sánh các kiến trúc trên dữ liệu dạng bảng

Tiêu chí	MLP	Random Forest	XGBoost	FT-Transformer
Tương tác đặc trưng tương minh	Không	Có	Có	Có
Fine-tuning chọn lọc (Layer Freezing)	Có	Không	Không	Có
Phù hợp đặc trưng liên tục nhiều	Trung bình	Tốt	Tốt	Rất tốt
Hỗ trợ mở rộng đặc trưng (Model Surgery)	Có	Không	Không	Có

2.4 Các kỹ thuật xử lý mất cân bằng dữ liệu

Mất cân bằng dữ liệu là thách thức đặc trưng của bài toán IDS. Khi tỷ lệ lớp thiểu số (tấn công) quá nhỏ so với lớp đa số (Benign), mô hình học máy bị kéo về phía tối ưu hóa cho lớp đa số và bỏ qua lớp thiểu số — đây chính xác là hành vi không mong muốn trong bối cảnh an ninh mạng, nơi bỏ sót một cuộc tấn công nghiêm trọng hơn nhiều so với báo nhầm một kết nối lành tính. Ba kỹ thuật chính được sử dụng trong đề tài được trình bày dưới đây.

2.4.1 Focal Loss

Hàm mất mát Cross-Entropy tiêu chuẩn cho bài toán đa lớp với C lớp được định nghĩa:

$$\mathcal{L}_{CE}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{c=1}^C y_c \log \hat{y}_c \quad (2.11)$$

trong đó $y_c \in \{0, 1\}$ là nhãn one-hot và $\hat{y}_c$ là xác suất dự đoán cho lớp c . Ký hiệu

p_t là xác suất dự đoán cho nhãn đúng (với bài toán nhị phân):

$$p_t = \begin{cases} \hat{y} & \text{nếu } y = 1 \\ 1 - \hat{y} & \text{nếu } y = 0 \end{cases} \quad (2.12)$$

Khi đó Cross-Entropy viết lại thành $\mathcal{L}_{CE} = -\log(p_t)$. Vấn đề cốt lõi với dữ liệu mất cân bằng: mẫu Benign dễ phân loại (có $p_t \approx 0,99$) vẫn đóng góp gradient $-\log(0,99) \approx 0,01$ vào quá trình tối ưu. Nhỏ nhưng khi có hàng triệu mẫu Benign, đóng góp gradient tổng hợp áp đảo hoàn toàn gradient từ vài nghìn mẫu tấn công thiểu số, khiến mô hình học để phân loại tốt Benign và bỏ qua các lớp nhỏ.

Focal Loss [12] giải quyết vấn đề này bằng cách thêm hệ số điều chỉnh $(1 - p_t)^\gamma$ để làm giảm đóng góp của các mẫu dễ:

$$\mathcal{L}_{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \quad (2.13)$$

Tham số $\gamma \geq 0$ là *tham số tập trung* (focusing parameter): khi $\gamma = 0$, Focal Loss rút gọn về Cross-Entropy thông thường. Khi $\gamma > 0$, hệ số $(1 - p_t)^\gamma$ downweight mạnh các mẫu dễ và giữ nguyên trọng số cho mẫu khó. Với $\gamma = 2$, mẫu có $p_t = 0,9$ chỉ đóng góp $(1 - 0,9)^2 = 0,01$ lần so với mẫu có $p_t = 0,5$ — giảm 100 lần.

Tham số $\alpha_t \in (0, 1)$ là hệ số cân bằng lớp, thường đặt tỷ lệ nghịch với tần suất lớp để bù thêm cho các lớp thiểu số.

2.4.2 SMOTE

SMOTE (Synthetic Minority Over-sampling Technique) [13] tạo ra các mẫu tổng hợp cho lớp thiểu số bằng nội suy tuyến tính trong không gian đặc trưng, thay vì chỉ nhân bản mẫu gốc (oversampling ngây thơ). Với mỗi mẫu thiểu số $\mathbf{x}_i$, SMOTE xác định K láng giềng gần nhất trong cùng lớp $\{\mathbf{x}_{k_1}, \dots, \mathbf{x}_{k_K}\}$ và tạo mẫu mới bằng nội suy ngẫu nhiên:

$$\mathbf{x}_{new} = \mathbf{x}_i + \lambda (\mathbf{x}_{k_r} - \mathbf{x}_i), \quad \lambda \sim \text{Uniform}(0, 1) \quad (2.14)$$

trong đó $\mathbf{x}_{k_r}$ là láng giềng được chọn ngẫu nhiên từ K láng giềng của $\mathbf{x}_i$. Mẫu mới $\mathbf{x}_{new}$ nằm trên đoạn thẳng nối $\mathbf{x}_i$ và $\mathbf{x}_{k_r}$, đảm bảo nằm trong vùng đặc trưng của lớp thiểu số và tạo ra sự đa dạng thực sự (không phải bản sao).

Phiên bản mở rộng **SMOTE-ENN** kết hợp SMOTE với Edited Nearest Neighbors (ENN): sau khi tạo mẫu tổng hợp, ENN loại bỏ những mẫu (cả tổng hợp lẫn gốc) bị phân loại sai bởi K -NN. Bước ENN làm sạch vùng biên giữa các

lớp, loại bỏ noise và các mẫu ngoại lai, giúp mô hình học biên giới quyết định rõ ràng hơn.

2.4.3 Weighted Random Sampling và Class Weights

Weighted Random Sampling điều chỉnh xác suất chọn mẫu trong mỗi mini-batch để tạo ra phân phối cân bằng hơn trong quá trình huấn luyện, mà không cần tạo thêm dữ liệu. Trọng số của mẫu i thuộc lớp c được tính:

$$w_i = \frac{N}{K \cdot n_c} \quad (2.15)$$

trong đó N là tổng số mẫu, K là số lớp, và n_c là số mẫu của lớp c . Tập $\{w_i\}$ được chuẩn hóa thành xác suất chọn mẫu, đảm bảo mỗi lớp xuất hiện với tần suất tương đương trong mỗi batch huấn luyện.

Để tránh trọng số cực đoan gây mất ổn định, thực tế thường dùng biến thể làm mượt $w_i = \sqrt{N/(K \cdot n_c)}$ và clip về khoảng $[w_{min}, w_{max}]$. Trong Giai đoạn 4 (Testbed V8.5), class weights được tính theo $\sqrt{N/(K \cdot n_c)}$ và clip về $[0,5; 2,0]$, cho BruteForce weight = 1,328 và PortScan weight = 1,587.

Ngoài Weighted Sampling tác động vào quá trình lấy mẫu batch, **Label Smoothing** tác động vào nhãn mục tiêu: thay vì one-hot cứng, nhãn được làm mềm bằng cách phân phối một phần nhỏ ε sang các lớp còn lại:

$$y_c^{smooth} = (1 - \varepsilon) y_c + \frac{\varepsilon}{C} \quad (2.16)$$

Label Smoothing với $\varepsilon = 0,10$ được sử dụng trong V8.5 để giảm overfitting của mô hình lên các pattern training cụ thể.

2.5 Học chuyển giao và thích nghi miền

Học chuyển giao (Transfer Learning) là kỹ thuật tái sử dụng kiến thức học được từ miền nguồn (source domain) để giải quyết bài toán trong miền đích (target domain). Trực giác căn bản là: các tầng thấp của mô hình học các biểu diễn chung (đặc trưng tổng quát), trong khi các tầng cao học biểu diễn đặc thù nhiệm vụ cụ thể.

Trong bối cảnh đề tài, miền nguồn là CIC-IDS-2017 (thu thập trên mạng switch vật lý, gigabit) và miền đích là Testbed WSL (thu thập qua NAT và card mạng ảo). Sự khác biệt hạ tầng tạo ra *Covariate Shift*: phân phối đặc trưng $P_{source}(\mathbf{x}) \neq P_{target}(\mathbf{x})$ dù hàm nhãn $P(y | \mathbf{x})$ về mặt lý thuyết không thay đổi. Chứng cứ định lượng: mô hình CIC áp dụng trực tiếp lên Testbed đạt $MCC = -0,015$ (tệ hơn cả

dự đoán ngẫu nhiên).

Kỹ thuật **Layer Freezing Fine-tuning** giải quyết covariate shift bằng cách đóng băng các tầng thấp của mô hình (tham số θ_{freeze} không được cập nhật) và chỉ tối ưu các tầng cao và đầu phân loại (tham số θ_{fine}):

$$\theta_{fine}^* = \arg \min_{\theta_{fine}} \mathcal{L}(f_{\theta_{freeze}, \theta_{fine}}(\mathbf{x}_{target}), y_{target}) \quad (2.17)$$

Rủi ro chính của fine-tuning là *Catastrophic Forgetting*: khi cập nhật quá nhiều tham số, mô hình có thể quên kiến thức từ miền nguồn. Chỉ số này được định lượng:

$$CF = \frac{\text{Acc}_{source, before} - \text{Acc}_{source, after}}{\text{Acc}_{source, before}} \times 100\% \quad (2.18)$$

Trong đề tài, chiến lược đóng băng 2 tầng Attention đầu và chỉ cập nhật 2 tầng cuối cùng với đầu phân loại đạt $CF = 0,61\%$ — trong ngưỡng chấp nhận được cho ứng dụng thực tế.

Model Surgery là kỹ thuật mở rộng kiến trúc mô hình có sẵn để hỗ trợ không gian đặc trưng mới mà không cần huấn luyện lại từ đầu. Khi Testbed yêu cầu thêm 3 đặc trưng kỹ thuật mới (IAT_CV, Bwd_Pkt_Ratio, Pkt_Size_Ratio), chiều đầu vào được mở rộng $77 \rightarrow 80$: ma trận embedding $\mathbf{W}^{emb} \in \mathbb{R}^{77 \times d}$ được mở rộng thành $\mathbb{R}^{80 \times d}$, với 77 hàng cũ giữ nguyên (transplant) và 3 hàng mới khởi tạo ngẫu nhiên để học trong giai đoạn fine-tuning.

2.6 Hệ thống IDS lai ghép: phát hiện theo luật và theo hành vi bất thường

Hệ thống IDS lai ghép kết hợp hai hướng tiếp cận bổ sung cho nhau, nhằm tận dụng điểm mạnh và bù đắp điểm yếu của từng phương pháp. Trong kiến trúc của đề tài, hai thành phần chính là Snort và FT-Transformer.

Snort [2] hoạt động ở tầng gói tin (packet-level), phân tích payload và header của từng gói tin theo thời gian thực với độ trễ cỡ microsecond. Với cơ sở dữ liệu luật được cập nhật thường xuyên bởi cộng đồng (Snort Community Rules, Emerging Threats), Snort phát hiện hiệu quả các tấn công đã biết với tỷ lệ false positive thấp khi luật được viết cẩn thận. Điểm yếu là không thể phát hiện tấn công chưa có luật, và không có khả năng phát hiện các tấn công khai thác hành vi (behavioral exploits) không để lại signature đặc trưng trong payload.

FT-Transformer hoạt động ở tầng flow, nhận đầu vào là vector đặc trưng CICFlowMeter được tổng hợp từ một kết nối hoàn chỉnh. Khả năng học pattern từ dữ liệu cho phép phát hiện các tấn công có hành vi bất thường ở mức thống kê

dù chưa được biết cụ thể. Điểm yếu là độ trễ cao hơn (cần đợi flow hoàn chỉnh) và nhạy cảm với covariate shift.

Trong pipeline End-to-End của đề tài, hai thành phần chạy song song và độc lập trên cùng luồng traffic:

- Snort lắng nghe trực tiếp trên interface mạng, tạo cảnh báo ngay khi phát hiện luật khớp với gói tin.
- CICFlowMeter thu thập PCAP từ cùng interface, tổng hợp flow và trích xuất vector đặc trưng 80 chiều sau khi flow hoàn chỉnh (timeout hoặc RST/FIN), đưa vào FT-Transformer để phân loại.
- Cảnh báo từ bất kỳ nguồn nào đều được ghi log và đẩy lên hệ thống cảnh báo thống nhất.

Sự kết hợp này đảm bảo: tấn công có luật Snort được phát hiện gần real-time ở tầng packet; tấn công dựa trên bất thường hành vi được phát hiện ở tầng flow. Đặc biệt quan trọng là hai thành phần hoạt động *độc lập* — lỗi hoặc sự vắng mặt của một thành phần không ảnh hưởng đến thành phần kia, đảm bảo tính sẵn sàng của hệ thống giám sát.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Tổng quan kiến trúc hệ thống

Hệ thống NIDS được đề xuất trong đề tài được xây dựng qua ba giai đoạn nghiên cứu tuần tự, mỗi giai đoạn được thúc đẩy bởi kết quả định lượng của giai đoạn trước: từ việc xác lập kiến trúc nền tảng trên NSL-KDD, tái thiết kế cho CIC-IDS-2017 với 9 lớp hiện đại, đến thu thập dữ liệu Testbed nội bộ và triển khai End-to-End.

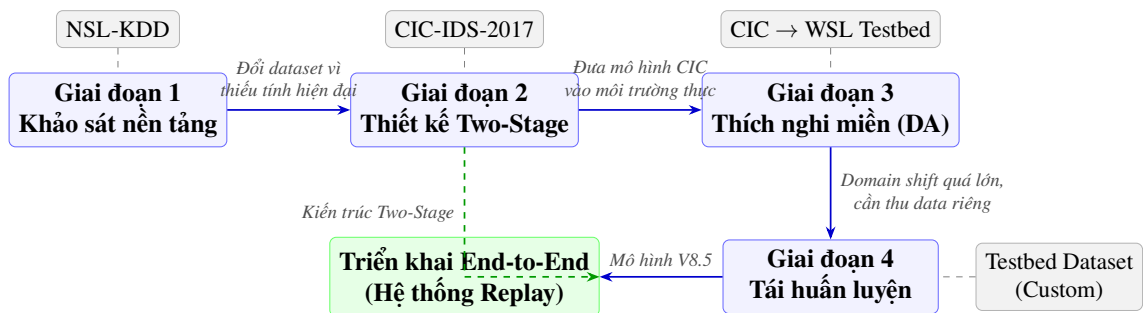
Giai đoạn 1 (Mục 3.2) xây dựng mô hình nhị phân và đa lớp trên NSL-KDD, thiết lập các kỹ thuật xử lý mất cân bằng và pipeline tiền xử lý. Giai đoạn này kết thúc bằng quyết định chuyển sang CIC-IDS-2017 với lý do định lượng rõ ràng.

Giai đoạn 2 (Mục 3.3) tái thiết kế kiến trúc hoàn toàn cho CIC-IDS-2017: Two-Stage Cascade (Gating Network 6-lớp + Expert Network 5-lớp, xuất 9 lớp tổng hợp từ 15 nhãn gốc) kết hợp Asymmetric Ensemble Voting, đạt Accuracy 99,55% và Macro F1 0,9294.

Giai đoạn 3 (Mục 3.4) giải quyết vấn đề covariate shift khi triển khai mô hình CIC sang môi trường Testbed WSL, thông qua Layer Freezing Fine-tuning và Model Surgery.

Giai đoạn 4 (Mục 3.5) tái huấn luyện hoàn toàn trên dữ liệu Testbed nội bộ, giải quyết các bài toán đặc thù của môi trường WSL và cho ra mô hình cuối V8.5 đạt Macro F1 91,7%.

Hệ thống cuối cùng (Mục 3.6) tích hợp Snort và FT-Transformer trong pipeline End-to-End chạy song song, thu thập traffic thực, trích xuất đặc trưng và phân loại theo thời gian gần-thực.



Hình 3.1: Tổng quan quá trình nghiên cứu: từ khảo sát nền tảng đến triển khai End-to-End thực tế. Mỗi giai đoạn giải quyết một giới hạn phát sinh từ giai đoạn trước.

3.2 Giai đoạn 1: Xây dựng mô hình nền tảng trên NSL-KDD

3.2.1 Đặc điểm bộ dữ liệu NSL-KDD và quy trình tiền xử lý

NSL-KDD [5] là bản cải tiến của bộ dữ liệu KDD Cup 1999 [7], được thiết kế để khắc phục một số vấn đề của bản gốc: loại bỏ các bản ghi trùng lặp (KDD Cup 1999 có tới 78% bản ghi lặp trong tập training), cân bằng lại tỷ lệ giữa tập train và test để đảm bảo kết quả đánh giá không bị ảnh hưởng bởi sự phân bố không đều. NSL-KDD gồm 125.973 bản ghi training và 22.544 bản ghi test với 4 lớp tấn công chính: DoS, Probe, R2L (Remote to Local), và U2R (User to Root), cùng nhãn con chi tiết hơn.

Dữ liệu NSL-KDD sử dụng 41 đặc trưng dạng hỗn hợp: 38 đặc trưng số (duration, byte counts, flag counts, v.v.) và 3 đặc trưng phân loại (protocol_type, service, flag). Quy trình tiền xử lý bao gồm: mã hóa one-hot cho đặc trưng phân loại, chuẩn hóa MinMaxScaler cho đặc trưng số, và xử lý mất cân bằng dữ liệu qua SMOTE-ENN cho các lớp thiểu số (U2R, R2L).

3.2.2 Kiến trúc Two-Stage: Autoencoder và Stacking Ensemble

Bài toán phân loại đa lớp trực tiếp trên NSL-KDD gặp phải một cái bẫy cấu trúc: nếu một mô hình duy nhất nhận toàn bộ 125.973 mẫu và phải phân loại năm lớp, gradient sẽ bị chi phối bởi Normal (53,5%) và DoS (36,5%), trong khi U2R — lớp quan trọng nhất về mặt bảo mật — chỉ có 52 mẫu training, tương đương 0,04%. Kiến trúc Two-Stage được thiết kế để tháo gỡ sự xung đột này bằng cách tách biệt hai bài toán có bản chất khác nhau.

Stage 1 — Autoencoder Anomaly Gate (Bộ lọc dị thường):

Autoencoder được huấn luyện *chỉ* trên dữ liệu Normal theo nguyên lý unsupervised: mạng encoder-decoder học cách tái tạo lại các flow bình thường với sai số thấp nhất. Khi một flow tấn công đi qua mạng này — có pattern khác hoàn toàn với những gì mô hình đã học — lỗi tái tạo sẽ cao hơn đáng kể.

Sai số tái tạo được đo bằng Mean Squared Error:

$$\text{MSE}(\mathbf{x}) = \frac{1}{D} \sum_{i=1}^D (x_i - \hat{x}_i)^2 \quad (3.1)$$

trong đó $D = 122$ là số chiều sau mã hóa one-hot, $\hat{x}$ là đầu ra tái tạo của Autoencoder.

Quyết định phân loại của Stage 1:

$$\text{gate}(\mathbf{x}) = \begin{cases} \text{Normal} & \text{MSE}(\mathbf{x}) < \tau \\ \text{Suspicious} & \text{MSE}(\mathbf{x}) \geq \tau \end{cases} \quad (3.2)$$

với ngưỡng $\tau = 0,008481$ được chọn trên tập validation để tối đa hóa F1 nhị phân. Các flow được gán nhãn Normal thoát khỏi pipeline; các flow Suspicious được chuyển sang Stage 2.

Lợi thế căn bản của Autoencoder Gate: mô hình học “không gian chuẩn” của traffic bình thường mà *không cần nhãn tấn công*. Bất kỳ flow nào bị tái tạo kém là flow bất thường theo định nghĩa, kể cả tấn công chưa từng xuất hiện trong training. Stage 1 đạt Macro-F1 nhị phân = 0,84 và Precision = 0,93 trên KDDTest+.

Stage 2 — Stacking Ensemble cho phân loại chi tiết:

Stage 2 nhận các flow Suspicious từ Stage 1 và phân loại thành 4 lớp tấn công (DoS, Probe, R2L, U2R). Thay vì một mô hình đơn lẻ, Stacking Ensemble kết hợp hai bộ phân loại có inductive bias khác nhau để bù đắp điểm yếu lẫn nhau.

Base learner 1 — FT-Transformer: $d = 128$, 8 heads, 4 lớp Attention, $d_{ff} = 512$, dropout = 0,1. Cơ chế Attention cho phép mô hình học tương tác giữa các cặp đặc trưng (ví dụ: mối quan hệ giữa `src_bytes` và `rerror_rate`) — hữu ích để phân biệt R2L với các tấn công khác.

Base learner 2 — LightGBM: 127 lá, learning rate = 0,05, $n_estimators = 1000$ (early stopping patience 50), subsample = 0,8. LightGBM phân vùng không gian đặc trưng theo cơ chế cây quyết định — bắt được các ngưỡng rõ ràng trong đặc trưng như `logged_in` hay `num_compromised`, đặc biệt hiệu quả cho Probe và U2R.

Hai mô hình này sai trên các mẫu khác nhau: FT-Transformer đạt R2L Recall cao hơn (0,383 vs 0,208 của LightGBM) nhờ cơ chế Attention; LightGBM đạt Probe F1 và U2R F1 cao hơn nhờ phân vùng đặc trưng cứng. Đây là điều kiện lý tưởng để Stacking phát huy.

Meta-learner — Logistic Regression: được huấn luyện trên đầu ra xác suất (out-of-fold predictions) của cả hai base learner:

$$\hat{y} = \text{Meta-LR}([\mathbf{p}_{\text{FTT}}; \mathbf{p}_{\text{LGBM}}]), \quad \mathbf{p}_{\text{FTT}}, \mathbf{p}_{\text{LGBM}} \in \mathbb{R}^5 \quad (3.3)$$

Vector đầu vào của Meta-LR có 10 chiều (xác suất 5 lớp từ mỗi base learner). Meta-LR học *khi nào* nên tin FT-Transformer và *khi nào* nên tin LightGBM hơn —

một dạng chuyên môn hóa động theo từng vùng không gian đặc trưng.

Kết quả đầu-cuối trên KDDTest+ (5 lớp, bao gồm cả Stage 1): Macro F1 = 0,6809, Accuracy = 80,09%. So với từng mô hình đơn lẻ (FT-Transformer: 0,654; LightGBM: 0,668), Stacking cải thiện đồng thời Normal F1 (0,823), Probe F1 (0,794) và U2R F1 (0,512).

3.2.3 Chiến lược xử lý mất cân bằng

NSL-KDD có hai dạng mất cân bằng về bản chất khác nhau, đòi hỏi hai chiến lược khác nhau:

Dạng 1 — Thiếu hụt dữ liệu (U2R): Chỉ có 52 mẫu U2R trong toàn bộ KDDTrain+. SMOTE không hiệu quả ở đây vì các mẫu tổng hợp được nội suy tuyến tính trong không gian 122 chiều với chỉ 52 điểm gốc — phần lớn là bản sao gần các mẫu gốc, không tăng thực sự sự đa dạng.

Dạng 2 — Dịch chuyển phân phối (R2L): R2L có 995 mẫu training (0,8%) nhưng 2.885 mẫu test (12,8%) — gấp 16 lần. Đồng thời, 62% test R2L sử dụng giao thức pop3 (không có trong training, vốn 100% là telnet). Bổ sung thêm R2L synthetic không giải quyết được vấn đề này vì mô hình thiếu exposure với pop3 patterns, không phải thiếu số lượng.

Các chiến lược được áp dụng theo tầng:

1. Chiến lược Val Set Boost-Minority: Thay vì chia ngẫu nhiên 15%, tập validation được xây dựng có chủ đích:

- R2L: 40% mẫu vào val — giữ sự đa dạng attack types trong validation để checkpoint selection phản ánh đúng hiệu năng trên R2L.
- U2R: chỉ 5 mẫu vào val, phần còn lại (47 mẫu) vào training — tăng 52% dữ liệu U2R training và đảm bảo checkpoint selection không bị dominated bởi 8 mẫu U2R không đại diện.
- Normal/DoS: 10% vào val.

2. Class-Balanced Focal Loss: Alpha cho mỗi lớp được tính theo công thức class-balanced:

$$\alpha_c = \frac{1 - \beta}{1 - \beta^{n_c}}, \quad \beta = 0,9999 \quad (3.4)$$

Kết hợp với $\gamma = 2$ của Focal Loss (xem phương trình 2.13), giá trị α_c lớn cho U2R/R2L đảm bảo gradient từ các lớp thiểu số không bị “chôn vùi” bởi khối lượng Normal/DoS.

3. WeightedRandomSampler (FT-Transformer): Mỗi batch được lấy mẫu với

xác suất tỷ lệ nghịch với tần suất lớp, đảm bảo mỗi batch training chứa đại diện của tất cả các lớp — kể cả U2R.

4. Inverse-Frequency Sample Weights (LightGBM): Mỗi mẫu training nhận trọng số $w_c = N/(K \cdot n_c)$ được đưa vào trực tiếp trong quá trình boosting, tăng đóng góp gradient của các mẫu thiểu số ở mỗi vòng xây cây.

Kết quả cuối của chiến lược tổng hợp: U2R Recall = 0,4776 (tăng từ 0,15 ở baseline không xử lý), R2L Precision = 0,9681. R2L Recall vẫn ở mức 0,25 — giới hạn cứng do distribution shift pop3/telnet không thể giải quyết bằng kỹ thuật sampling hay loss, chỉ có thể giải quyết bằng thêm dữ liệu R2L đa dạng giao thức hơn.

3.2.4 Giới hạn của NSL-KDD và động cơ chuyển sang CIC-IDS-2017

Sau khi đạt kết quả ổn định trên NSL-KDD, hai giới hạn căn bản của bộ dữ liệu này trở nên rõ ràng. Thứ nhất, dữ liệu được thu thập từ năm 1998 trong một môi trường mạng giả lập — không phản ánh bất kỳ loại tấn công hiện đại nào: không có DDoS amplification, không có web application attacks (SQLi, XSS), không có botnet C2 communication, không có brute force qua SSH/FTP. Kết quả trên NSL-KDD, dù cao về mặt metric, không có giá trị thực tiễn cho bài toán IDS hiện đại.

Thứ hai, NSL-KDD chỉ có 4 lớp tấn công bậc cao (DoS, Probe, R2L, U2R) — không đủ chi tiết cho hệ thống IDS cần phân biệt giữa DDoS và DoS, hay giữa PortScan và brute force. Môi trường thực tế đòi hỏi phân loại tấn công ở mức độ chi tiết hơn để hỗ trợ đội ngũ SOC phản ứng đúng cách với từng loại sự cố.

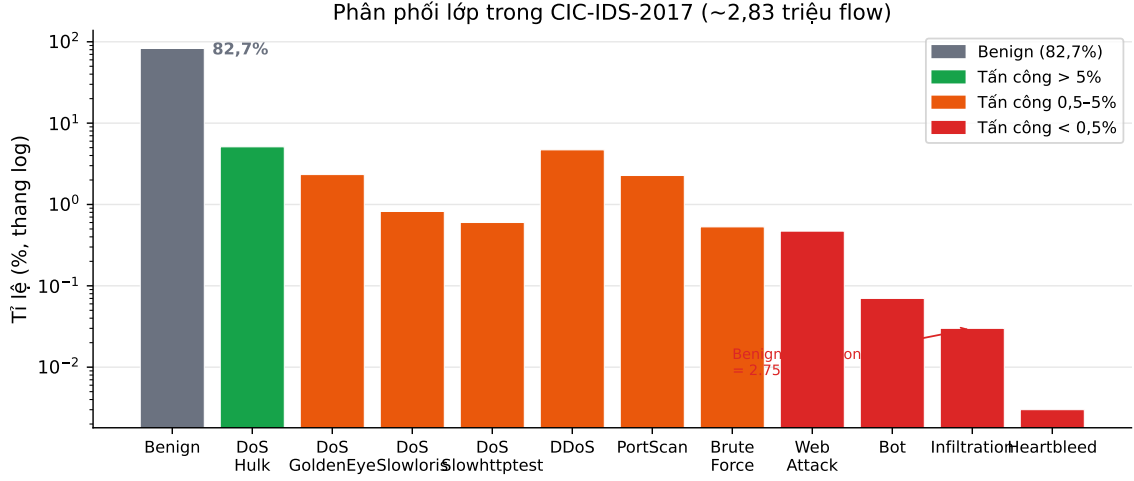
CIC-IDS-2017 [3] giải quyết cả hai vấn đề: dữ liệu thu thập năm 2017 với traffic thực từ 25 users thực hiện các kịch bản tấn công hiện đại qua 5 ngày, với 15 nhãn gốc (Benign và 14 loại tấn công) được gộp thành 9 lớp mô hình (Benign, DoS, DDoS, BruteForce, PortScan, Web Attack, Botnet, Infiltration, Heartbleed). Đây là cơ sở để chuyển sang Giai đoạn 2.

3.3 Giai đoạn 2: Two-Stage Cascade NIDS trên CIC-IDS-2017

3.3.1 Bộ dữ liệu CIC-IDS-2017 và thách thức mất cân bằng cực đoan

CIC-IDS-2017 [3] gồm traffic thu thập trong 5 ngày liên tiếp: thứ Hai (chỉ Benign), thứ Ba đến thứ Sáu (các kịch bản tấn công). Sau khi xử lý, tập dữ liệu sử dụng trong đề tài gồm khoảng 2,83 triệu flow với phân phối lớp mất cân bằng nghiêm trọng: lớp Benign chiếm 82,7%, trong khi các lớp tấn công dao động từ DoS Slowloris (0,23%) đến Heartbleed (0,003%). Ba lớp đặc biệt thách thức là Botnet (0,07%), Infiltration (0,03%) và Heartbleed (0,003%) — tỷ lệ Benign:Infiltration lên đến hơn 2.750:1.

Thách thức không chỉ nằm ở tỷ lệ mà còn ở sự *không đồng đều về mức độ khó phân tích*: Heartbleed có pattern flow rất đặc trưng (dù hiếm) nên dễ học, trong khi Botnet có traffic C2 bắt chước Benign và Infiltration thực hiện exfiltration chậm không tạo pattern flow rõ ràng.



Hình 3.2: Phân phối lớp trong CIC-IDS-2017 (2,83 triệu flow, thang logarithm). Benign chiếm 82,7%; tỉ lệ Benign:Infiltration vượt 2.750:1.

3.3.2 Quy trình tiền xử lý và kỹ thuật hóa đặc trưng

Dữ liệu thô từ CICFlowMeter chứa một số vấn đề chất lượng cần xử lý trước khi huấn luyện. Các giá trị vô cực (Infinity) và NaN xuất hiện trong một số đặc trưng tốc độ (Flow Bytes/s, Flow Packets/s) khi duration của flow bằng 0 — những flow này được xử lý bằng cách clip về giá trị lớn nhất hợp lệ (cho Infinity) hoặc loại bỏ (cho NaN không thể impute).

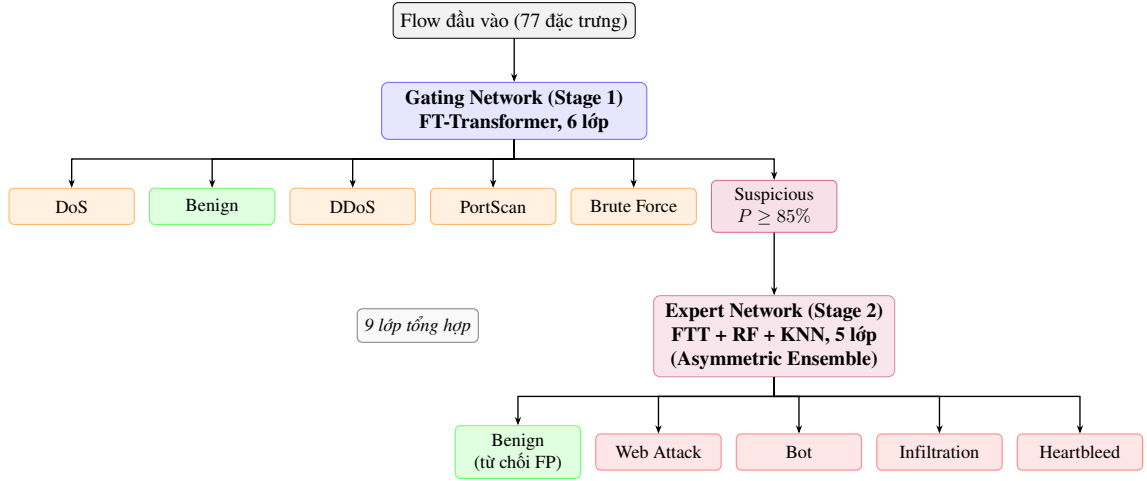
Chuẩn hóa đặc trưng sử dụng **PowerTransformer** với **phân phối Yeo-Johnson** [14]. Phân phối Yeo-Johnson tổng quát hóa Box-Cox để xử lý được cả giá trị âm, với tham số λ được ước lượng bằng Maximum Likelihood để tối đa hóa tính xấp xỉ Gaussian của phân phối sau biến đổi:

$$\psi_{\lambda}(x) = \begin{cases} \frac{(x+1)^{\lambda} - 1}{\lambda} & \lambda \neq 0, x \geq 0 \\ \ln(x+1) & \lambda = 0, x \geq 0 \\ -\frac{[(-x+1)^{2-\lambda} - 1]}{2-\lambda} & \lambda \neq 2, x < 0 \\ -\ln(-x+1) & \lambda = 2, x < 0 \end{cases} \quad (3.5)$$

Lý do chọn PowerTransformer thay vì StandardScaler hay MinMaxScaler: nhiều đặc trưng CICFlowMeter có phân phối power-law (byte counts, IAT) với đuôi dài

cực đoan. StandardScaler không thay đổi phân phối; MinMaxScaler đưa về $[0, 1]$ nhưng không giải quyết skewness. PowerTransformer đưa phân phối về xấp xỉ Gaussian, giúp FT-Transformer học hiệu quả hơn và đặc biệt giúp các lớp thiểu số không bị “chôn vùi” trong đuôi phân phối của Benign.

Ngoài ra, một số đặc trưng có tương quan Pearson cao ($r > 0,95$) được loại bỏ để tránh multicollinearity, giảm chiều từ 77 xuống 34 đặc trưng cho Expert Network (Stage 2).



Hình 3.3: Kiến trúc Two-Stage Cascade NIDS: Gating Network (6 lớp) lọc các tấn công phổ biến và chuyển Suspicious flows sang Expert Network (5 lớp). Hệ thống xuất 9 lớp tổng hợp từ 15 nhãn gốc.

3.3.3 Tầng 1: Gating Network

Gating Network là bộ phân loại **6 lớp**: nhận 77 đặc trưng đầy đủ và phân loại flow thành {Benign, DoS, DDoS, PortScan, Brute Force, Suspicious}. Bốn lớp tấn công phổ biến (DoS, DDoS, PortScan, Brute Force) có pattern traffic đặc trưng rõ ràng và được phân loại trực tiếp tại Stage 1. Lớp thứ sáu — Suspicious — bao gồm các tấn công phức tạp (Web Attack, Bot, Infiltration, Heartbleed) có flow-level signature gần giống Benign; các flow này được chuyển tiếp sang Expert Network. Thiết kế 6-lớp có hai lợi ích: (1) Stage 1 phân loại sẵn 4 nhóm tấn công dễ nhận biết, giải phóng Expert Network để tập trung vào các tấn công khó; (2) Expert Network không phải xử lý toàn bộ traffic — chỉ nhận các flow Suspicious.

Gating Network là một FT-Transformer với cấu hình ($d = 128$, 4 lớp Attention, 8 đầu, $d_{ff} = 512$, dropout=0,2) được huấn luyện với CB-Focal Loss ($\gamma = 2$) và Weighted Sampling để ưu tiên recall tấn công. Ngưỡng 85% được áp dụng cho xác suất lớp Suspicious: chỉ khi $P(\text{Suspicious}) \geq 0,85$ thì flow mới được chuyển sang Expert Network; nếu Stage 1 dự đoán Suspicious nhưng $P < 0,85$, flow được gán nhãn Benign (bảo thủ, tránh gánh nặng xử lý thêm).

3.3.4 Tầng 2: Expert Network và Hard Negative Mining

Expert Network nhận đầu vào là các flow được Gating Network đánh dấu Suspicious và phân loại thành **5 lớp**: Benign, Web Attack, Bot, Infiltration, Heartbleed. Nhãn Benign trong Stage 2 cho phép Expert Network từ chối các flow bị Gating Network chuyển nhầm (false positive Stage 1). Toàn bộ hệ thống xuất **9 lớp**: Stage 1 xác định trực tiếp Benign/DoS/DDoS/PortScan/Brute Force; Stage 2 phân loại thêm Web Attack, Bot, Infiltration, Heartbleed cho các flow Suspicious. Mô hình sử dụng 34 đặc trưng (tập con sau loại bỏ tương quan cao) — giảm chiều giúp Expert Network tập trung vào các đặc trưng thực sự phân biệt các tấn công phức tạp.

Hard Negative Mining (HNM) được thực hiện hai vòng. Sau vòng huấn luyện đầu, toàn bộ tập validation được đưa qua Expert Network và các mẫu bị phân loại sai được xác định. Những mẫu này — đặc biệt là Botnet và Infiltration bị nhầm thành Benign hoặc các lớp tấn công khác — được **nhân bản 10 lần** và bổ sung vào tập huấn luyện vòng tiếp theo, tạo áp lực học đủ lớn trên các mẫu khó. Sau hai vòng HNM, Botnet F1 cải thiện từ 0,4787 lên 0,6512, và Infiltration F1 cải thiện từ 0,3821 lên 0,5903.

3.3.5 Từ Expert Network đơn lẻ đến Asymmetric Ensemble Voting

Sau hai vòng HNM, Expert Network đạt kết quả tốt cho hầu hết các lớp nhưng vẫn còn hai điểm yếu đặc thù: Botnet F1 dưới 0,65 và Infiltration F1 dưới 0,60. Phân tích confusion matrix cho thấy hai nguyên nhân khác nhau:

Với **Botnet**: FT-Transformer có xu hướng nhầm một số Benign traffic thành Botnet (false positive cao). Nguyên nhân: pattern Botnet C2 heartbeat ở tần suất thấp trùng với một số Benign long-idle connection. Random Forest và KNN — với inductive bias khác — ít bị lỗi này hơn.

Với **Infiltration**: FT-Transformer bỏ sót nhiều Infiltration (false negative cao). Nguyên nhân: các flow Infiltration riêng lẻ gần giống Benign SSH/HTTP. Tuy nhiên, các lớp thiểu số cực đoan thường được Random Forest học tốt hơn nhờ cơ chế bootstrap sampling trong từng cây. KNN cũng có xu hướng phân loại Infiltration dựa trên lân cận cục bộ, tốt hơn với các lớp có vùng đặc trưng nhỏ, cô lập.

Kết luận: ba mô hình sai trên những mẫu khác nhau — điều kiện lý tưởng cho ensemble. Ensemble sẽ chỉ sai khi đa số mô hình đồng thời sai, xác suất thấp hơn nhiều so với mô hình đơn lẻ.

3.3.6 Thiết kế Asymmetric Ensemble Voting

Ba mô hình thành phần của Ensemble: (1) FT-Transformer (Expert Network, đã mô tả ở trên); (2) Random Forest với 150 cây, max_depth=25, max_features=20, class_weight=balanced; (3) KNN với K=16, trọng số đều (uniform). Mỗi mô hình được huấn luyện độc lập trên cùng tập dữ liệu (sau HNM).

Luật bỏ phiếu tiêu chuẩn là majority voting: nhãn được bầu bởi ít nhất 2/3 mô hình. Hai luật **bất đối xứng** bổ sung được thiết kế dựa trên phân tích chi phí lỗi:

- **Quy tắc Ưu tiên Infiltration:** Nếu RF *hoặc* KNN bỏ phiếu Infiltration, kết quả cuối là Infiltration — bất kể FT-Transformer bỏ phiếu gì. Lý do: Infiltration chiếm dưới 0,03% dữ liệu và bỏ sót một cuộc Infiltration có hậu quả nghiêm trọng hơn rất nhiều so với một false alarm Infiltration. Quy tắc này tăng Infiltration Recall đáng kể đổi lấy một phần Precision.
- **Quy tắc Đồng thuận Botnet:** Kết quả là Botnet *chỉ khi* FT-Transformer dự đoán Botnet và cả RF lẫn KNN đều không dự đoán Benign. Lý do: FT-Transformer thường xuyên nhầm Benign thành Botnet (FP cao), gây alert fatigue. RF và KNN ít bị lỗi FP Botnet hơn nhờ inductive bias khác; nếu một trong hai dự đoán Benign thì kết quả cuối bị override thành Benign.

Kết quả Ensemble cuối: Accuracy 99,55%, Macro F1 0,9294, với Botnet F1 cải thiện lên 0,7344 và Infiltration F1 lên 0,7407 so với Expert Network đơn lẻ (xem Chi tiết ở Chương 5).

3.4 Giai đoạn 3: Thách nghiệm miền sang Testbed thực chiến

3.4.1 Chẩn đoán Covariate Shift

Sau khi mô hình CIC đạt kết quả tốt, bước tiếp theo là triển khai lên môi trường Testbed thực của đề tài: một máy tính cá nhân chạy Windows 11 với WSL2 Ubuntu, kết nối mạng qua card mạng ảo (Hyper-V Virtual Ethernet Adapter) và NAT. Traffic tấn công được tạo ra từ một thiết bị khác trên cùng mạng LAN và gửi đến IP của WSL host.

Khi áp dụng mô hình CIC trực tiếp lên 5.000 flow thu thập từ Testbed, kết quả là Accuracy 21,93% và MCC = -0,015. Đây là mức tệ hơn cả dự đoán ngẫu nhiên theo MCC — cho thấy mô hình không chỉ hoạt động kém mà còn đang phân loại *sai hướng* một cách có hệ thống. Nguyên nhân được xác định qua phân tích phân phối đặc trưng:

Hai môi trường tạo ra phân phối đặc trưng khác nhau đáng kể do:

- **MTU và frame size khác nhau:** Switch vật lý Gigabit xử lý frame đầy đủ;

card mạng ảo Hyper-V có overhead ảo hóa thay đổi packet size và timing.

- **NAT làm thay đổi flow statistics:** NAT của Windows thêm độ trễ và thay đổi timing của ACK, ảnh hưởng đến các đặc trưng IAT (Inter-Arrival Time) và flag counts.
- **PowerTransformer parameters không phù hợp:** Scaler được fit trên phân phối CIC (switch vật lý, gigabit) — khi áp dụng lên phân phối Testbed (card ảo, NAT), một số đặc trưng bị map sai miền giá trị.

3.4.2 Tại sao Re-fit Scaler thất bại

Giải pháp đơn giản nhất được thử đầu tiên: giữ nguyên kiến trúc mô hình, chỉ fit lại PowerTransformer trên phân phối Testbed và chuẩn hóa lại đầu vào. Kết quả: Accuracy giảm từ 21,93% xuống còn 6,87% — tệ hơn nhiều.

Lý do: mô hình FT-Transformer đã học các biểu diễn nội bộ (embedding của từng feature token) dựa trên khoảng giá trị của CIC-scaler. Khi scaler thay đổi, đầu vào sau chuẩn hóa có phân phối giá trị hoàn toàn khác — các token embedding mapping này không còn ý nghĩa nữa. Nói cách khác, Re-fit Scaler thay đổi “ngôn ngữ” đầu vào của mô hình trong khi mô hình vẫn “nói” ngôn ngữ cũ — tạo ra sự không tương thích còn tệ hơn ban đầu.

Bài học: khi fine-tune mô hình cho miền mới, scaler phải được thay đổi *cùng lúc* với ít nhất một phần tham số mô hình — không thể thay đổi một trong hai một cách độc lập.

3.4.3 Fine-tuning với Layer Freezing

Layer Freezing Fine-tuning được thiết kế để thích nghi mô hình với scaler mới và phân phối Testbed mà không phá vỡ kiến thức đã học từ CIC. Chiến lược cụ thể:

1. Fit lại PowerTransformer trên tập Testbed (thay scaler).
2. Đóng băng tham số của 2 tầng Attention đầu — các tầng học đặc trưng tổng quát như mối quan hệ cơ bản giữa duration/byte/packet counts.
3. Chỉ cập nhật tham số của 2 tầng Attention cuối và đầu phân loại (classification head) — các tầng học biểu diễn đặc thù miền.
4. Huấn luyện trên tập Testbed với learning rate nhỏ (10^{-4}) và Mixed-Domain Batch: mỗi batch chứa 70% Testbed flows + 30% CIC flows để ngăn Catastrophic Forgetting.

Kết quả sau Layer Freezing: MCC tăng từ $-0,015$ lên $0,6825$, Accuracy đạt 99,82%. Catastrophic Forgetting $CF = 0,61\%$ — mô hình vẫn giữ 99,39% hiệu năng trên CIC ban đầu.

3.4.4 Model Surgery: mở rộng không gian đặc trưng

Phân tích phân phối đặc trưng Testbed so với CIC cho thấy một số đặc trưng có thể cải thiện khả năng phân biệt trong môi trường WSL nhưng không có trong tập 77 đặc trưng của pipeline CIC. Ba đặc trưng kỹ thuật mới được thiết kế và tính toán từ dữ liệu CICFlowMeter:

- **Custom_IAT_CV** (Coefficient of Variation của Inter-Arrival Time): $IAT_CV = \sigma_{IAT} / (\mu_{IAT} + \varepsilon)$ — đo tính đều đặn của timing gói tin; Botnet C2 heartbeat có CV thấp (periodic), Benign browsing có CV cao (irregular).
- **Custom_Bwd_Pkt_Ratio**: Tỷ lệ gói tin backward trên forward: $N_{bwd} / (N_{fwd} + \varepsilon)$ — PortScan SYN scan gần như không có gói backward (≈ 0), Benign HTTP có tỷ lệ cân bằng.
- **Custom_Pkt_Size_Ratio**: Tỷ lệ kích thước gói tin nhỏ nhất trên lớn nhất: $L_{min} / (L_{max} + \varepsilon)$ — DoS flood có gói kích thước đồng đều (tỷ lệ gần 1), Benign có phân phối đa dạng (tỷ lệ thấp).

Model Surgery mở rộng chiều đầu vào $77 \rightarrow 80$: ma trận Feature Tokenizer Embedding $W^{emb} \in \mathbb{R}^{77 \times d}$ được mở rộng thành $\mathbb{R}^{80 \times d}$. 77 hàng cũ được giữ nguyên (transplant) và 3 hàng mới được khởi tạo ngẫu nhiên $\mathcal{N}(0, 0.01)$. Các tham số còn lại của mô hình không thay đổi. Sau đó, fine-tuning với Layer Freezing được chạy lại để học embedding của 3 đặc trưng mới.

Sau Model Surgery, MCC tăng thêm từ 0,6825 lên 0,7333, với cải thiện chủ yếu ở Botnet (Custom_IAT_CV giúp phân biệt periodic heartbeat) và PortScan (Custom_Bwd_Pkt_Ratio tiết lộ tính một chiều của SYN scan).

3.5 Giai đoạn 4: Tái huấn luyện mô hình trên dữ liệu Testbed nội bộ

3.5.1 Đặc điểm bộ dữ liệu Testbed nội bộ

Sau khi domain adaptation từ Giai đoạn 3 giải quyết được một phần vấn đề covariate shift, thực nghiệm cho thấy mô hình vẫn có những điểm yếu đặc thù của môi trường WSL không thể khắc phục bằng fine-tuning. Quyết định được đưa ra: thu thập dữ liệu Testbed nội bộ và tái huấn luyện hoàn toàn mô hình trên dữ liệu này.

Testbed gồm hai máy tính trên cùng mạng LAN: (1) máy Windows 11 (IP 192.168.1.x) chạy WSL2 Ubuntu là máy victim — cài đặt Snort, CICFlowMeter, và DVWA (Damn Vulnerable Web Application) cho web attacks; (2) máy tấn công Kali Linux chạy các công cụ tấn công thực (nmap, hydra, slowhttptest, curl-based web attacks). WSL2 sử dụng chế độ Mirrored Networking, cho phép traffic từ mạng vật lý đến trực tiếp WSL interface mà không qua NAT translation hoàn toàn.

Dữ liệu Testbed nội bộ thu thập được gồm 5 lớp: Benign (lưu lượng sinh hoạt thực tế), BruteForce (hydra SSH/FTP), DoS (slowhttptest, hping3), PortScan (nmap), và Web Attack (curl-based SQLi, XSS). Không thu thập Infiltration và Botnet do không thể mô phỏng đáng tin cậy trên môi trường hai máy.

3.5.2 Bài toán PortScan: tại sao điều chỉnh thuật toán không đủ

PortScan là thách thức đầu tiên và nghiêm trọng nhất của dữ liệu Testbed nội bộ. Kết quả ban đầu (V8.2): PortScan F1 = 7,5% — mức thực tế bằng không. Mọi thử nghiệm điều chỉnh thuật toán — tăng class weight, dùng Focal Loss với γ cao, thêm SMOTE — đều không cải thiện đáng kể (xem phân tích chi tiết tại Mục 4.2).

Nguyên nhân gốc rễ: trên môi trường WSL với NAT, mỗi lần chạy nmap từ máy tấn công chỉ tạo ra khoảng 7 flow được CICFlowMeter capture, thay vì hàng trăm flow như trên switch vật lý. Mỗi flow này là một TCP connection rất ngắn, không có payload — hoàn toàn không phân biệt được với Benign health check hay short-lived HTTP connection.

Giải pháp: inject 5.000 flow PortScan từ CIC-IDS-2017 Friday dataset (được thu thập trên switch vật lý thực) vào tập huấn luyện. Các flow CIC PortScan có đặc trưng rõ ràng — nhiều SYN flags, duration ngắn nhưng *đồng nhất*, byte count gần bằng 0 — đủ phân biệt với Benign. Đây là giải pháp *dữ liệu surrogate*: dùng dữ liệu từ môi trường khác (CIC real hardware) để bù cho dữ liệu thiếu chất lượng ở môi trường hiện tại (WSL).

Kết quả sau injection CIC PortScan: PortScan F1 tăng từ 7,5% lên 100% — và duy trì ổn định qua mọi phiên bản tiếp theo.

3.5.3 Bài toán Brute Force: thiếu đa dạng pattern và giải pháp tăng cường dữ liệu

Sau khi giải quyết PortScan, thách thức tiếp theo là BruteForce. Dữ liệu nội bộ (Run10) chỉ gồm hydra SSH/FTP attacks — một loại tool duy nhất, tạo ra một pattern flow duy nhất (TCP connections liên tiếp với authentication timeout). BruteForce F1 trên tập validation nội bộ ban đầu đạt 70,3%, với Recall chỉ 54% — nghĩa là bỏ sót gần 1 trong 2 cuộc tấn công brute force.

Để mở rộng độ đa dạng của dữ liệu BruteForce, CIC Patator flows (FTP-Patator và SSH-Patator, 4.999 flows từ CIC Tuesday) được thêm vào tập huấn luyện. Patator sử dụng cơ chế authentication bruteforce khác hydra — tốc độ thử khác nhau, pattern TCP connection khác nhau — giúp mô hình học dạng tổng quát hơn của BruteForce, không chỉ học pattern của hydra.

3.5.4 Vấn đề đồng nhất miền trong tập kiểm định và hệ quả lên độ tin cậy của metric

Sau khi thêm CIC Patator vào tập huấn luyện, tập validation được cấu thành chỉ từ CIC Patator flows (500 flows, 10% stratified split). Kết quả: BruteForce F1 = 100%, Macro F1 = 94,95%.

Con số 100% này đáng ngờ — và thực tế là không đáng tin cậy. Tập validation gồm 100% CIC Patator flows trong khi training đã thấy CIC Patator flows — đây là đánh giá in-distribution, không phản ánh khả năng tổng quát hóa sang hydra (tool khác, pattern khác). Một mô hình overfit hoàn toàn vào Patator pattern cũng sẽ đạt F1 = 100% trên tập validation này.

Để kiểm tra thực sự, tập validation được xây dựng lại với thành phần *đa dạng miền*: 400 hydra flows (từ Run10) + 398 Patator flows (CIC) = 798 flows BruteForce tổng cộng. Kết quả trên tập validation đa dạng này: BruteForce P=83%, R=90%, F1=86%; Macro F1 = 91,7%. Con số thấp hơn nhưng đáng tin cậy hơn — phản ánh khả năng phát hiện BruteForce từ cả hai nguồn tool.

Đây là minh chứng trực tiếp: metric cao trên tập validation đồng nhất miền không đảm bảo hiệu năng tốt trong thực tế với traffic đa dạng hơn.

3.5.5 Mô hình cuối V8.5: thiết kế, huấn luyện và đánh giá

Mô hình cuối V8.5 được fine-tune từ mô hình tốt nhất trước đó, với tập dữ liệu Combined_V8_5 gồm 111.825 flows:

Bảng 3.1: Thành phần tập dữ liệu huấn luyện V8.5

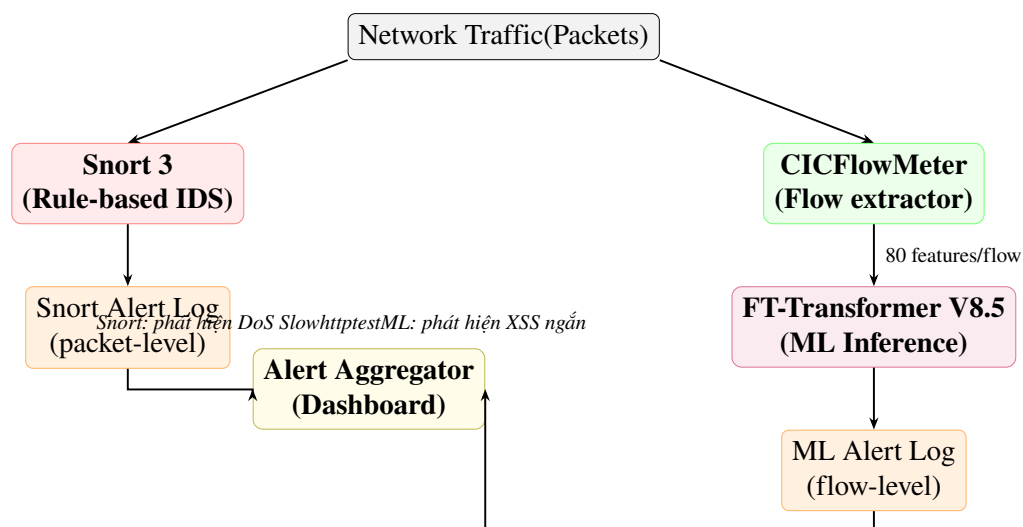
Lớp	Nguồn	Số flows	Tỷ lệ
Benign	Testbed nội bộ (nhiều runs)	51.975	46,5%
DoS	Testbed nội bộ	19.800	17,7%
Web Attack	Run10 + CIC Thursday XSS/SQLi	27.067	24,2%
BruteForce	Run10 hydra + CIC Patator	7.983	7,1%
PortScan	CIC Friday	5.000	4,5%
Tổng		111.825	100%

Cấu hình huấn luyện: learning rate theo từng lớp (backbone 10^{-5} , mid $1,5 \times 10^{-5}$, head 3×10^{-5}), dropout 0,15, label smoothing 0,10, weight decay 2×10^{-4} , class weights $\sqrt{N/(K \cdot n_c)}$ clip về [0,5; 2,0]. Early stopping với patience=5, dừng tại epoch 6 (best checkpoint: epoch 1).

Kết quả đánh giá trên tập validation 11.183 flows: Macro F1 = 91,7%, Balanced Accuracy = 91,1%. Chi tiết per-class: Benign F1=91%, BruteForce F1=86% (P=83%/R=90%), DoS F1=93%, PortScan F1=100%, Web Attack F1=88%.

3.6 Triển khai Hybrid IDS End-to-End

3.6.1 Pipeline tích hợp Snort, CICFlowMeter và FT-Transformer



Hình 3.4: Kiến trúc mục tiêu của hệ thống Hybrid IDS End-to-End: Snort (tầng packet, rule-based) và FT-Transformer V8.5 (tầng flow, ML) chạy song song, alert được hợp nhất tại Alert Aggregator. Đường xử lý thời gian thực hoàn toàn là thiết kế mục tiêu; bản đã hiện thực và đánh giá là hệ thống phát hiện theo phát lại (Mục 3.6.2).

Kiến trúc mục tiêu của hệ thống End-to-End gồm bốn thành phần chạy song song trên máy victim (Hình 3.4):

Snort chạy ở chế độ IDS (không inline) trên interface mạng WSL, lắng nghe toàn bộ traffic. Ruleset sử dụng kết hợp bộ luật cộng đồng (Snort Community Rules, Emerging Threats Open) và đặc biệt là **bộ luật tự thiết kế (Custom Rules)** được tinh chỉnh riêng cho các công cụ tấn công trên Testbed (như Nmap PortScan, Hydra SSH/FTP, Slowloris, Hulk, SQLmap). Khi một gói tin khớp luật, Snort ghi alert vào log file với timestamp, source/dest IP/port, và tên luật kích hoạt. Snort hoạt động ở mức gói tin, xử lý real-time với độ trễ cỡ microsecond.

CICFlowMeter chạy trong chế độ capture liên tục, đọc packets từ interface WSL. Mỗi khi một flow hoàn chỉnh (TCP connection đóng bằng RST/FIN hoặc timeout 30 giây), CICFlowMeter tính toán 80 đặc trưng thống kê và ghi vào file CSV. Một process Python giám sát file CSV này và đẩy các dòng mới vào hàng đợi phân loại.

FT-Transformer Inference Engine đọc từ hàng đợi, áp dụng PowerTransformer (Testbed scaler) và đưa vào mô hình V8.5 để phân loại. Kết quả (nhãn lớp + confidence score) được ghi vào alert log thứ hai với thông tin flow đầy đủ.

Alert Aggregator đọc cả hai alert logs (Snort + ML) và hợp nhất vào một dashboard. Khi cùng một flow/session xuất hiện trong cả hai nguồn cảnh báo, mức

độ ưu tiên được tăng lên (confirmed alert).

Trong bốn thành phần trên, đường xử lý thời gian thực hoàn toàn — bắt gói liên tục từ interface, lắp ghép flow trực tuyến và đưa qua mô hình ngay khi flow đóng — đòi hỏi một dịch vụ truyền dòng ổn định mà phạm vi đồ án chưa hiện thực hóa trọn vẹn. Phần được xây dựng, kiểm thử và đánh giá đầy đủ là hệ thống phát hiện theo phát lại (replay-based live detection): thay vì bắt gói trực tiếp, hệ thống phát lại các tệp flow đã trích xuất sẵn qua đúng mô hình V8.5, đúng bộ tiền xử lý và đúng logic quyết định mà phiên bản thời gian thực sẽ sử dụng. Chỉ tầng thu gói và lắp ghép flow ở đầu vào được thay bằng việc đọc lại flow có sẵn; toàn bộ thành phần phía sau giữ nguyên. Cách tiếp cận này cho phép đánh giá định lượng hành vi phân loại của hệ thống trên dữ liệu Testbed thật, đồng thời tách biệt phần kỹ thuật truyền dòng còn dang dở. Việc hoàn thiện đường thời gian thực được đề xuất như một hướng phát triển tại Mục 5.

3.6.2 Hệ thống phát hiện theo phát lại

Hệ thống replay nạp mô hình V8.5 một lần, đọc tệp flow CICFlowMeter (*_Flow.csv) của từng kịch bản tấn công, phân loại toàn bộ flow rồi phát kết quả ra giao diện qua WebSocket theo nhịp mô phỏng thời gian thực, có điều khiển play, pause và tốc độ. Nhãn đối chứng được gán theo IP nguồn: flow xuất phát từ máy tấn công nhận nhãn loại tấn công của phiên thu, các flow còn lại nhận nhãn Benign. Gán nhãn theo IP loại bỏ sự mơ hồ của việc ghép cảnh báo theo cửa sổ thời gian giữa hai nguồn dữ liệu.

Trên đầu ra của mô hình, hệ thống áp một chuỗi luật hậu xử lý theo thứ tự cố định. Luật thứ nhất là ngưỡng tin cậy: một dự đoán thuộc lớp tấn công nhưng có xác suất cực đại dưới ngưỡng T được hạ về Benign. Với $T = 0,6$, luật cắt phần lớn cảnh báo giả sinh ra từ các dự đoán lưỡng lự mà không chạm tới các dự đoán tấn công có độ tin cậy cao. Luật thứ hai bù cho một đặc trưng mà bộ 80 chiều của V8.5 không chứa — mức độ quét cổng: với mỗi IP nguồn, hệ thống đếm số cổng đích duy nhất tiếp xúc trong một cửa sổ ngắn (mặc định 2 giây), và nếu số này vượt ngưỡng (15 cổng) với flow xuất phát từ máy quét, nhãn được ghi đè thành PortScan. Mỗi luật là một đơn vị độc lập theo nguyên tắc Open/Closed — thêm luật mới không sửa luật đã có.

Luật quét cổng là hiện thân trực tiếp của nguyên lý lai ghép: một heuristic kiểu dấu hiệu — đếm cổng đích theo nguồn — bù chính xác điểm mù của mô hình thống kê, vốn không phân biệt được PortScan với DoS khi cả hai biểu hiện thành nhiều flow ngắn. Kết quả định lượng của chuỗi luật này được trình bày tại Mục 5.6.

3.6.3 Môi trường giả lập tấn công xuyên thiết bị

Môi trường thực nghiệm End-to-End gồm hai máy trên cùng mạng LAN gia đình:

- **Máy victim** (Windows 11 + WSL2 Ubuntu): IP 192.168.1.x trên mạng LAN. WSL2 cấu hình Mirrored Networking để share IP với Windows host. Cài đặt: Apache2 + DVWA (cổng 80), OpenSSH Server (cổng 22), FTP Server (vsftpd, cổng 21). Snort, CICFlowMeter và FT-Transformer chạy bên trong WSL2.
- **Máy tấn công** (Kali Linux laptop): kết nối cùng WiFi router với máy victim. Các công cụ sử dụng: nmap (PortScan), hydra (BruteForce SSH/FTP), slowhttptest/hping3 (DoS), curl scripts (Web Attack: SQLi và XSS payload vào DVWA).

Quy trình thực nghiệm End-to-End: (1) khởi động Snort, CICFlowMeter và Inference Engine trên máy victim; (2) từ máy tấn công, thực hiện lần lượt từng loại tấn công theo kịch bản có kiểm soát; (3) ghi nhận alert từ cả hai nguồn (Snort và ML) và đối chiếu với nhãn tấn công thực tế; (4) đánh giá detection rate, false alarm rate và thời gian phát hiện.

Kết quả thực nghiệm End-to-End được trình bày chi tiết tại Mục 5.6.

CHƯƠNG 4. PHÂN TÍCH MÔ HÌNH

4.1 Phân tích Focal Loss trong điều kiện mất cân bằng cực đoan

Để hiểu tại sao Focal Loss vượt trội hơn Cross-Entropy tiêu chuẩn trong bài toán IDS, cần phân tích cơ chế gradient. Trong quá trình huấn luyện, mô hình cập nhật tham số theo gradient của hàm mất mát tổng hợp trên một mini-batch. Với Cross-Entropy, gradient đóng góp của từng mẫu tỷ lệ thuận với $-\log(p_t)$. Xét một batch gồm 1000 mẫu Benign để phân loại ($p_t = 0,95$) và 10 mẫu BruteForce khó ($p_t = 0,4$):

$$\text{Tổng gradient Benign} \propto 1000 \times (-\log 0,95) \approx 1000 \times 0,051 = 51 \quad (4.1)$$

$$\text{Tổng gradient BruteForce} \propto 10 \times (-\log 0,4) \approx 10 \times 0,916 = 9,16 \quad (4.2)$$

Dù mỗi mẫu BruteForce mang gradient lớn gấp 18 lần mỗi mẫu Benign, tổng gradient từ 1000 mẫu Benign vẫn áp đảo 5:1. Mô hình bị kéo liên tục về phía tối ưu cho Benign và ít có cơ hội học từ các mẫu tấn công thiểu số.

Với Focal Loss ($\gamma = 2$), gradient đóng góp được nhân với $(1 - p_t)^2$:

$$\text{Tổng gradient Benign (FL)} \propto 1000 \times (1 - 0,95)^2 \times 0,051 \approx 0,128 \quad (4.3)$$

$$\text{Tổng gradient BruteForce (FL)} \propto 10 \times (1 - 0,4)^2 \times 0,916 \approx 3,30 \quad (4.4)$$

Tỷ lệ đảo ngược: BruteForce giờ đóng góp gradient lớn gấp 25 lần Benign. Mô hình được buộc phải ưu tiên học từ các mẫu khó, khó phân loại — đúng là những mẫu cần học nhất trong bài toán IDS.

Tuy nhiên, Focal Loss không phải giải pháp toàn năng. Trong thực nghiệm Giai đoạn 4 của đề tài, Focal Loss giúp cải thiện BruteForce recall nhưng không giải quyết được vấn đề PortScan (F1 chỉ đạt 7,5% bất kể siêu tham số γ và α_t). Lý do: vấn đề PortScan không phải là *bất cân bằng gradient* (có thể giải quyết bằng Focal Loss) mà là *tính không phân tách đặc trưng* — hai lớp PortScan và Benign không thể phân biệt được trong không gian đặc trưng CICFlowMeter trên môi trường WSL, bất kể hàm mất mát nào được sử dụng.

4.2 Phân tích tính phân tách đặc trưng PortScan và Benign

Một lớp tấn công có thể phân tách (linearly separable hoặc separable theo mô hình phi tuyến) nếu tồn tại một hàm quyết định $f(x)$ phân biệt được các mẫu của hai lớp với accuracy đủ cao. Ngược lại, nếu hai lớp có vùng đặc trưng chồng lên nhau đáng kể, không có thuật toán học máy nào có thể phân tách chúng một cách đáng tin cậy.

Trong môi trường phần cứng thực (CIC-IDS-2017), PortScan (nmap) tạo ra hàng trăm flow TCP SYN ngắn đến nhiều cổng khác nhau trên cùng host đích — tạo ra pattern rõ ràng: Duration rất ngắn, Flow Bytes/s gần bằng 0, FIN/ACK flags thấp, SYN flag cao. Các đặc trưng này phân biệt rõ ràng với Benign HTTP/HTTPS traffic.

Trong môi trường WSL với NAT, tình huống hoàn toàn khác. Khi nmap chạy từ máy tính khác gửi SYN đến IP của WSL host, gói tin đi qua stack NAT của Windows trước khi đến WSL interface. Kết quả là CICFlowMeter trên WSL chỉ capture được khoảng 7 flow mỗi round nmap thay vì hàng trăm flow. Mỗi flow PortScan trong môi trường WSL là một kết nối TCP thông thường, rất ngắn, không có payload — không khác gì một Benign TCP health check hay một failed connection. Bằng chứng định lượng từ thực nghiệm:

Bảng 4.1: F1-score PortScan qua các thử nghiệm thuật toán và siêu tham số khác nhau (V8.2)

Cấu hình	PortScan F1	Nhận xét
FT-Transformer, CE Loss, class_weight=1.0	7,5%	Baseline
FT-Transformer, Focal Loss $\gamma = 2$	6,8%	Không cải thiện
FT-Transformer, class_weight PortScan $\times 5$	8,1%	Không cải thiện đáng kể
Random Forest, 150 cây	5,2%	Cũng thất bại
KNN, K=5	4,9%	Cũng thất bại

Kết quả nhất quán thất bại qua nhiều thuật toán khác nhau là bằng chứng mạnh cho tính không phân tách: vấn đề nằm ở dữ liệu, không phải mô hình. Giải pháp duy nhất hiệu quả là thay đổi ở tầng dữ liệu — inject dữ liệu PortScan được thu thập trên phần cứng thực (CIC Friday PortScan) làm surrogate, với các đặc trưng flow phản ánh đúng pattern của PortScan trong điều kiện thu thập đầy đủ.

Bài học tổng quát: khi nhiều thuật toán khác nhau đều thất bại trên cùng một lớp, nguyên nhân gần như chắc chắn là chất lượng hoặc tính đại diện của dữ liệu thu thập, không phải năng lực của mô hình.

4.3 Cơ sở lý thuyết của Hard Negative Mining

Hard Negative Mining (HNM) là kỹ thuật học tập trung vào các mẫu khó — những mẫu nằm gần biên giới quyết định và bị phân loại sai hoặc với confidence thấp. Về mặt lý thuyết, HNM liên hệ chặt chẽ với tư tưởng của Boosting: thay vì huấn luyện đồng đều trên tất cả mẫu, tập trung tài nguyên huấn luyện vào những mẫu mà mô hình hiện tại xử lý kém nhất.

Quá trình HNM trong đề tài được thực hiện theo hai vòng lặp. Sau vòng huấn luyện đầu (Stage 1 → Stage 2 Expert Network), toàn bộ tập validation được đưa qua mô hình và các mẫu bị phân loại sai được trích xuất. Những mẫu này — *hard negatives* — được thêm vào tập huấn luyện cho vòng tiếp theo với trọng số tăng cường. Mô hình buộc phải học từ chính những trường hợp nó đang xử lý sai.

Tại sao HNM hiệu quả hơn chỉ tăng class weight? Class weight tác động đồng đều lên toàn bộ lớp — mọi mẫu của lớp Botnet đều được tăng cường như nhau, kể cả những mẫu đã được phân loại đúng dễ dàng. HNM tập trung vào *vùng biên giới quyết định*: chỉ những mẫu mà mô hình thực sự đang nhầm lẫn mới được tăng cường, cho phép mô hình tinh chỉnh biên giới quyết định một cách hiệu quả hơn. Trong thực nghiệm Giai đoạn 2, HNM cải thiện Botnet F1 từ 0,4787 lên 0,6512 — mức tăng 17,3% tuyệt đối sau hai vòng; kết hợp thêm với Asymmetric Ensemble Voting, kết quả cuối đạt 0,7344.

Giới hạn của HNM: kỹ thuật này hiệu quả khi hai lớp *có thể phân tách* nhưng biên giới quyết định chưa được học tốt. Với PortScan/Benign trong môi trường WSL — hai lớp không thể phân tách — HNM không giúp được vì không tồn tại biên giới quyết định nào phân tách chúng một cách đáng tin cậy.

4.4 Lý thuyết Ensemble Voting và kiểm soát Variance

Lý thuyết Ensemble dựa trên phân tích Bias-Variance của lỗi tổng quát hóa. Với một mô hình đơn lẻ f , lỗi dự đoán có thể phân tích:

$$\mathbb{E}[(y - f(\mathbf{x}))^2] = \underbrace{(\mathbb{E}[f(\mathbf{x})] - y)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(f(\mathbf{x}) - \mathbb{E}[f(\mathbf{x})])^2]}_{\text{Variance}} + \sigma_{\text{noise}}^2 \quad (4.5)$$

Với M mô hình độc lập $f_1, f_2, \dots, f_M$ có cùng phương sai σ^2 và cùng bias, mô hình ensemble trung bình $\bar{f} = \frac{1}{M} \sum_i f_i(\mathbf{x})$ có phương sai:

$$\text{Var}(\bar{f}) = \frac{\sigma^2}{M} + \frac{M-1}{M} \rho \sigma^2 \quad (4.6)$$

trong đó ρ là hệ số tương quan trung bình giữa các mô hình. Khi $\rho \rightarrow 0$ (các mô hình độc lập hoàn toàn), phương sai giảm xuống σ^2/M . Kết luận quan trọng: *để ensemble hiệu quả, các mô hình thành phần phải có sự đa dạng — sai trên những mẫu khác nhau.*

Đây chính là lý do chọn bộ ba FT-Transformer + Random Forest + KNN trong đề tài. Ba mô hình có *inductive bias* hoàn toàn khác nhau:

- FT-Transformer học tương tác đặc trưng qua cơ chế Attention theo cách global.
- Random Forest xây dựng quyết định từ các tập con đặc trưng ngẫu nhiên, tốt với ranh giới phi tuyến cục bộ.
- KNN phân loại dựa trên lân cận khoảng cách, nhạy cảm với mật độ cục bộ.

Ba mô hình có xu hướng sai trên những mẫu khác nhau — đảm bảo ρ thấp, tức là ensemble giảm được variance đáng kể.

Asymmetric Voting là sự mở rộng của majority voting tiêu chuẩn khi không phải tất cả lỗi đều có chi phí như nhau. Hai luật bất đối xứng được thiết kế dựa trên đặc thù của từng lớp:

- **Infiltration Priority Rule:** Nếu RF hoặc KNN bỏ phiếu cho Infiltration, kết quả cuối là Infiltration — bất kể FT-Transformer nghĩ gì. Lý do: Infiltration rất hiếm (dưới 0,1% tập dữ liệu) và bỏ sót Infiltration có hậu quả nghiêm trọng hơn nhiều so với false alarm. Luật này tăng recall cho Infiltration đổi lấy một ít precision.
- **Botnet Consensus Rule:** Kết quả là Botnet chỉ khi cả ba mô hình đều bỏ phiếu Botnet. Lý do: Botnet pattern rất đa dạng trong CIC-IDS-2017 (nhiều biến thể C2 communication) và FT-Transformer thường nhầm Benign traffic thành Botnet. Yêu cầu đồng thuận tăng precision đổi lấy một ít recall.

Hai luật này phản ánh sự hiểu biết domain-specific về chi phí lỗi, không phải tối ưu toán học thuần túy.

4.5 Cơ sở của Layer Freezing trong Transfer Learning

Cơ sở lý thuyết của Layer Freezing xuất phát từ quan sát thực nghiệm trong cộng đồng deep learning: các tầng thấp của mạng nơ-ron sâu học các biểu diễn tổng quát (low-level features), trong khi các tầng cao học biểu diễn đặc thù nhiệm vụ (task-specific features) [15]. Điều này đã được xác nhận trực quan qua việc trực quan hóa filter trong CNN: các filter tầng đầu học edge detector và texture, trong khi các filter tầng sâu học hình dạng đối tượng cụ thể.

Với FT-Transformer trong bối cảnh IDS, phân tầng này có ý nghĩa cụ thể:

- Các tầng Attention đầu học các mẫu tương tác đặc trưng tổng quát: duration ngắn thường đi với byte count thấp; SYN flood thường đi với FIN flag thấp. Những mẫu này là bất biến theo môi trường thu thập.
- Các tầng Attention sâu học phân biệt tinh tế giữa các lớp tấn công cụ thể, phụ thuộc nhiều hơn vào phân phối đặc trưng của môi trường cụ thể.

Khi đóng băng 2 tầng Attention đầu và chỉ cập nhật 2 tầng cuối + đầu phân loại, mô hình giữ lại kiến thức tổng quát từ CIC (đã học trên 2,8 triệu flow) trong khi thích nghi các biểu diễn cấp cao với phân phối Testbed. Catastrophic Forgetting 0,61% cho thấy chiến lược này thành công trong việc cân bằng *plasticity* (khả năng học mới) và *stability* (giữ lại kiến thức cũ) — hai đại lượng luôn xung đột nhau trong học chuyển giao.

Nếu toàn bộ tầng được mở đóng băng, mô hình overfits nhanh chóng vào Testbed (ít dữ liệu hơn) và quên pattern từ CIC — đây là nguyên nhân Re-fit Scaler thất bại: scaler mới thay đổi phân phối đặc trưng đầu vào đột ngột, khiến toàn bộ biểu diễn học được trở nên vô nghĩa.

4.6 Giới hạn của dữ liệu flow-based tĩnh: trường hợp Infiltration

Tấn công Infiltration là trường hợp giới hạn căn bản của hướng tiếp cận phát hiện dựa trên đặc trưng flow tĩnh. Để hiểu tại sao, cần phân tích bản chất của Infiltration trong CIC-IDS-2017: kẻ tấn công xâm nhập vào mạng nội bộ thông qua một lỗ hổng ứng dụng, sau đó thực hiện lateral movement và data exfiltration theo tốc độ chậm, cố ý bắt chước lưu lượng bình thường.

Đặc trưng CICFlowMeter mô tả từng flow theo kiểu tĩnh — một snapshot của kết nối 5-tuple trong một khoảng thời gian. Với Infiltration, mỗi flow riêng lẻ trông như một kết nối hợp lệ: một HTTP GET nhỏ, một DNS query bình thường, một SSH session ngắn. Pattern bất thường chỉ lộ ra khi quan sát *chuỗi* các flow theo thời gian: tần suất kết nối đến server bên ngoài tăng bất thường, các DNS query đến domain lạ, hay lượng data upload lớn bất thường vào đêm khuya.

Đây là vấn đề phân tích dữ liệu chuỗi thời gian (temporal analysis), không phải phân loại một điểm dữ liệu đơn lẻ. Các mô hình phân loại flow đơn lẻ — bao gồm cả FT-Transformer — về nguyên lý không thể nắm bắt thông tin ngữ cảnh này. Kết quả trong đề tài xác nhận: dù đã áp dụng Hard Negative Mining và Ensemble Voting, Infiltration F1 chỉ đạt 0,7407 trên CIC-IDS-2017 — một trong những kết quả thấp nhất trong 9 lớp.

Giải pháp tiềm năng cho vấn đề Infiltration nằm ngoài phạm vi đề tài hiện tại nhưng đáng được đề cập: session aggregation (tổng hợp nhiều flow của cùng một

session thành một vector đặc trưng dài hơn), temporal graph analysis (phân tích đồ thị kết nối theo thời gian), hoặc payload inspection kết hợp với flow-level analysis. Đây là hướng phát triển tự nhiên tiếp theo cho hệ thống.

Giới hạn tương tự cũng xuất hiện với Botnet trong giai đoạn C2 communication: các flow C2 riêng lẻ trông như Benign DNS/HTTP, nhưng pattern periodic heartbeat chỉ rõ khi quan sát nhiều flow liên tiếp. Đây là lý do Botnet F1 dù đã cải thiện đáng kể ($0,4787 \rightarrow 0,7344$ nhờ HNM) vẫn là lớp có kết quả thấp nhất trong Stage 2.

CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM

5.1 Môi trường thực nghiệm, bộ dữ liệu và tham số đánh giá

5.1.1 Môi trường phần cứng và phần mềm

Toàn bộ thực nghiệm được thực hiện trên một máy tính cá nhân với cấu hình phổ thông, không có GPU chuyên dụng — phản ánh điều kiện phần cứng thực tế mà đề tài hướng tới khả năng triển khai:

- **Phần cứng:** CPU Intel Core i7 thế hệ 11, RAM 16 GB DDR4, ổ SSD. Toàn bộ quá trình huấn luyện chạy trên CPU với PyTorch CPU backend.
- **Hệ điều hành:** Windows 11 (host) với WSL2 Ubuntu 22.04 LTS. WSL2 cấu hình chế độ Mirrored Networking để chia sẻ địa chỉ IP với Windows host, cho phép traffic từ mạng LAN vật lý tiếp cận trực tiếp interface WSL.
- **Thư viện chính:** Python 3.8, PyTorch 2.0 (FT-Transformer), TensorFlow/Keras 2.12 (Autoencoder), LightGBM 4.0, scikit-learn 1.3, imbalanced-learn 0.11.
- **Công cụ mạng:** CICFlowMeter v4 (trích xuất đặc trưng flow từ live interface, timeout 30 giây), Snort 3.1 (IDS rule-based, Custom Rules + Community Rules + Emerging Threats Open).

5.1.2 Bộ dữ liệu thực nghiệm

Ba bộ dữ liệu được sử dụng qua từng giai đoạn nghiên cứu, phản ánh quá trình dịch chuyển từ benchmark chuẩn sang dữ liệu thu thập thực chiến:

Bảng 5.1: Tổng quan các bộ dữ liệu thực nghiệm

Dataset	Giai đoạn	Tổng mẫu	Đặc trưng	Số lớp
NSL-KDD Train+	1	125.973	41 gốc → 122	5
NSL-KDD Test+	1	22.542	41 gốc → 122	5
CIC-IDS-2017	2, 3	2.830.743	77	15
Testbed V8.5	4	111.825	80	5

NSL-KDD sử dụng 41 đặc trưng gốc, được mở rộng lên 122 chiều sau mã hóa one-hot cho ba đặc trưng phân loại (`protocol_type`: 3 giá trị, `service`: 70 giá trị, `flag`: 11 giá trị) và `MinMaxScaler` cho phần còn lại. CIC-IDS-2017 sử dụng 77 đặc trưng — gồm 68 đặc trưng thống kê CICFlowMeter, 5 chỉ báo nhóm cổng và 4 đặc trưng tỉ lệ được kỹ thuật hóa trong pipeline tiền xử lý — chuẩn hóa bằng `PowerTransformer` (Yeo-Johnson). Testbed V8.5 mở rộng tập này lên 80 đặc trưng bằng 3 đặc trưng từ bước Model Surgery (`IAT_CV`, `Bwd_Pkt_Ratio`, `Pkt_Size_Ratio`), chuẩn hóa bằng `PowerTransformer` được fit trên

phân phối Testbed.

5.1.3 Các chỉ số đánh giá

Tất cả các bài toán phân loại trong đề tài đều có mất cân bằng lớp đáng kể, khiến accuracy trở thành chỉ số không đáng tin cậy — một mô hình dự đoán tất cả là Benign cũng đạt trên 80% accuracy trên CIC-IDS-2017. Ba chỉ số chính được sử dụng:

Macro-F1 là metric chính để so sánh mô hình, trong đó mỗi lớp được trọng số bằng nhau, phản ánh yêu cầu bảo mật: bỏ sót một cuộc tấn công U2R (52 mẫu training) nghiêm trọng tương đương bỏ sót DoS (45.927 mẫu training):

$$\text{Macro-F1} = \frac{1}{K} \sum_{c=1}^K \frac{2P_c R_c}{P_c + R_c} \quad (5.1)$$

MCC (Matthews Correlation Coefficient) được dùng cho đánh giá domain adaptation vì nhạy hơn với hiệu năng đồng thời trên tất cả các lớp khi phân phối bất đối xứng cực đoan. Giá trị MCC = 0 tương đương dự đoán ngẫu nhiên; MCC âm cho thấy mô hình đang phân loại sai hướng có hệ thống.

Balanced Accuracy (= trung bình Recall qua tất cả các lớp) được dùng bổ sung cho Testbed để phản ánh rõ hiệu năng trên từng lớp thiểu số.

5.2 Kết quả giai đoạn NSL-KDD

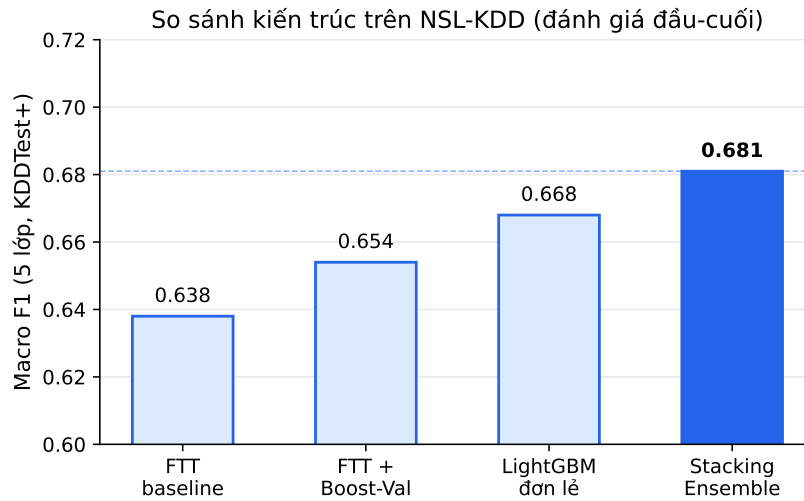
Giai đoạn NSL-KDD thiết lập kiến trúc Two-Stage (Autoencoder gate + Stage 2 Stacking Ensemble) và xác định các chiến lược xử lý mất cân bằng cốt lõi. Tất cả kết quả được đánh giá trên KDDTest+ (22.542 mẫu) theo metric Macro-F1 trên 5 lớp đầu-cuối — tức là bao gồm cả ảnh hưởng của Stage 1 Autoencoder Gate lên kết quả cuối.

So sánh các phiên bản kiến trúc

Bảng 5.2 trình bày quá trình cải thiện kiến trúc từ mô hình đơn lẻ đến Stacking Ensemble cuối:

Bảng 5.2: So sánh Macro-F1 đầu-cuối (5 lớp, KDDTest+) qua các phiên bản kiến trúc

Kiến trúc	Macro F1	Ghi chú
FT-Transformer đơn lẻ (baseline)	0,638	Không tối ưu val set
FT-Transformer + Boost-Minority Val	0,654	Best FT-Transformer
LightGBM đơn lẻ	0,668	Probe/U2R tốt hơn; R2L kém hơn
Stacking Ensemble (FTT + LGBM → Meta-LR)	0,681	Kết quả cuối



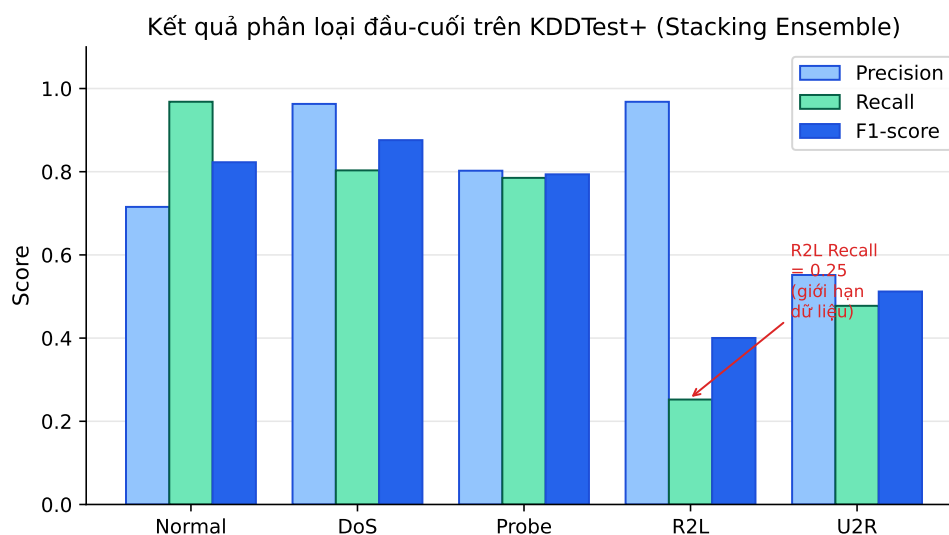
Hình 5.1: So sánh Macro F1 qua các phiên bản kiến trúc trên NSL-KDD (đánh giá đầu-cuối trên KDDTest+).

Mỗi bước cải thiện có động cơ rõ ràng: Boost-Minority Val giải quyết vấn đề checkpoint selection trên U2R (chỉ 52 training samples); LightGBM khai thác phân vùng đặc trưng cứng hiệu quả cho Probe; Stacking Ensemble kết hợp điểm mạnh của cả hai — FT-Transformer xử lý tốt R2L (Recall cao hơn nhờ Attention cross-feature), LightGBM xử lý tốt Probe và U2R.

Kết quả chi tiết per-class của kiến trúc cuối

Bảng 5.3: Kết quả phân loại đầu-cuối trên KDDTest+ (Two-Stage: AE Gate + Stacking Ensemble)

Lớp	Precision	Recall	F1	Support
Normal	0,7154	0,9682	0,8228	9.711
DoS	0,9630	0,8032	0,8759	7.458
Probe	0,8024	0,7852	0,7937	2.421
R2L	0,9681	0,2523	0,4003	2.885
U2R	0,5517	0,4776	0,5120	67
Macro avg	0,8001	0,6573	0,6809	22.542
Accuracy			0,8009	22.542



Hình 5.2: Kết quả phân loại đầu-cuối trên KDDTest+ — Stacking Ensemble (FTT + LightGBM → Meta-LR).

Phân tích kết quả

Điểm mạnh: Normal Recall đạt 0,9682 — Stage 1 Autoencoder Gate lọc lưu lượng bình thường với Precision binary = 0,93, chuyển 82,7% traffic Benign ra khỏi pipeline trước Stage 2. DoS F1 = 0,8759 và Probe F1 = 0,7937 phản ánh các lớp này có pattern flow đặc trưng và đủ dữ liệu training. U2R F1 = 0,5120 với chỉ 52 mẫu training là kết quả cạnh tranh so với literature.

Điểm yếu cấu trúc — R2L: R2L Recall = 0,2523 là giới hạn khó vượt qua. Phân tích cho thấy 62% test R2L bị Stage 1 AE phân loại nhầm thành Normal (MSE thấp hơn threshold) vì các flow này có tính chất timing giống Benign. Ngay cả khi những flow này đến được Stage 2, phần lớn R2L test sử dụng giao thức pop3 (61% test) trong khi toàn bộ R2L training là telnet — model không có khả năng generalize sang protocol chưa thấy. Đây là giới hạn cứng của NSL-KDD, không phải của thuật toán.

Đặt trong bối cảnh literature: Macro-F1 = 0,6809 trên KDDTest+ (5 lớp, 5-way end-to-end, không rõ rí train/test) nằm trong khoảng cạnh tranh 0,60–0,70 của các nghiên cứu đánh giá trung thực. Các kết quả báo cáo 99%+ trên NSL-KDD trong literature thường sử dụng cross-validation trên KDDTrain+ — không phản ánh distribution shift thật giữa KDDTrain+ và KDDTest+.

Kết quả này xác nhận hai điều: kiến trúc Two-Stage với Stacking Ensemble là hướng đúng cho bài toán mất cân bằng nhiều lớp; và giới hạn trên của NSL-KDD (dữ liệu 1998, thiếu diversity protocol) không thể vượt qua bằng cải tiến thuật toán đơn thuần — động cơ trực tiếp để chuyển sang CIC-IDS-2017 với dữ liệu hiện đại và phong phú hơn.

5.3 Kết quả Two-Stage Cascade NIDS trên CIC-IDS-2017

5.3.1 Hiệu quả kiến trúc Two-Stage so với mô hình 1-Stage

Vấn đề cốt lõi với mô hình 1-Stage trực tiếp trên CIC-IDS-2017: Benign chiếm 82,7% trong 2,83 triệu flow. Khi một FT-Transformer đơn lẻ phải phân loại 9 lớp đồng thời (Benign và 8 nhóm tấn công gộp từ 15 nhãn gốc), gradient bị dominated bởi Benign và các lớp lớn, khiến các lớp tấn công hiếm như Infiltration (0,03%) và Botnet (0,07%) nhận gradient quá nhỏ để học được pattern phân biệt.

Kiến trúc Two-Stage tách bài toán thành hai không gian riêng biệt:

Stage 1 — Gating Network (6 lớp): Phân loại flow thành {Benign, DoS, DDoS, PortScan, Brute Force, Suspicious} trên 77 đặc trưng. Bốn lớp tấn công phổ biến được phân loại trực tiếp. Ngưỡng bảo thủ $P(\text{Suspicious}) \geq 85\%$ đảm bảo gần như không có tấn công phức tạp nào bị lọc nhầm; flow Suspicious dưới ngưỡng này được gán Benign.

Stage 2 — Expert Network (5 lớp): Chỉ nhận Suspicious flows từ Stage 1 và phân loại thành {Benign, Web Attack, Bot, Infiltration, Heartbleed}. Khi các lớp đa số (Benign, DoS, DDoS, PortScan, Brute Force) đã được lọc ra ngoài, Expert Network học trong không gian sạch hơn — gradient tập trung phân biệt *giữa các tấn công phức tạp* thay vì bị overwhelmed bởi lưu lượng đa số.

Lợi ích định lượng của phân tầng thể hiện rõ nhất ở hai lớp thiểu số cực đoan: Infiltration và Botnet. Khi Expert Network không còn phải “cạnh tranh” gradient với 82,7% Benign, Macro F1 tăng từ 0,7831 (1-Stage) lên 0,8817 (Two-Stage, trước HNM) — cải thiện 9,9 điểm tuyệt đối.

5.3.2 Đóng góp của Hard Negative Mining

Sau vòng huấn luyện đầu tiên của Expert Network, hai lớp tấn công vẫn có hiệu năng thấp: Botnet $F1 = 0,4787$ và Infiltration $F1 = 0,3821$. Phân tích confusion matrix cho thấy hai nguyên nhân khác nhau:

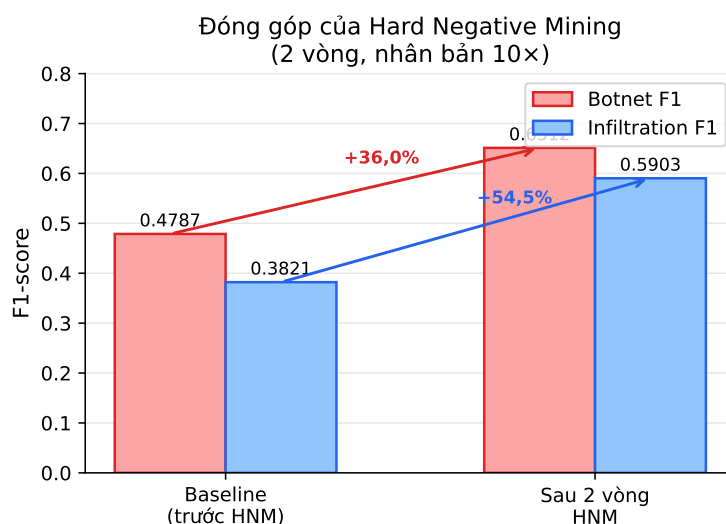
- **Botnet:** Nhiều false positive — nhầm một phần Benign long-idle connection thành Botnet C2 heartbeat do cả hai đều có pattern periodic, low-rate ở tần suất tương đương.
- **Infiltration:** Nhiều false negative — bỏ sót Infiltration bị phân loại sai thành Benign SSH/HTTP vì flow Infiltration riêng lẻ không có dấu hiệu bất thường rõ ràng ở mức per-flow.

Hard Negative Mining (HNM) chạy hai vòng: thu thập các mẫu bị phân loại sai trên tập validation, thêm vào training với trọng số tăng cường $w_{\text{hard}} = 2,0$, huấn

luyện lại. Bảng 5.4 cho thấy kết quả tích lũy:

Bảng 5.4: Kết quả Hard Negative Mining qua hai vòng trên hai lớp khó nhất

Giai đoạn	Botnet F1	Infiltration F1
Baseline Expert Network (trước HNM)	0,4787	0,3821
Sau 2 vòng Hard Negative Mining	0,6512	0,5903
Mức cải thiện	+0,1725 (+36,0%)	+0,2082 (+54,5%)



Hình 5.3: Cải thiện F1 của Botnet và Infiltration sau 2 vòng Hard Negative Mining (nhân bản 10×).

HNM hiệu quả vì nó tập trung tài nguyên học vào đúng những mẫu mà mô hình đang nhầm, thay vì học lại trên toàn bộ dataset (phần lớn là mẫu đã phân loại đúng). Sau 2 vòng HNM, Infiltration và Botnet cải thiện lần lượt 54,5% và 36,0% — xác nhận chiến lược curriculum này phù hợp với các lớp tấn công có pattern phức tạp, dễ nhầm.

5.3.3 Kết quả Ensemble Voting cuối cùng

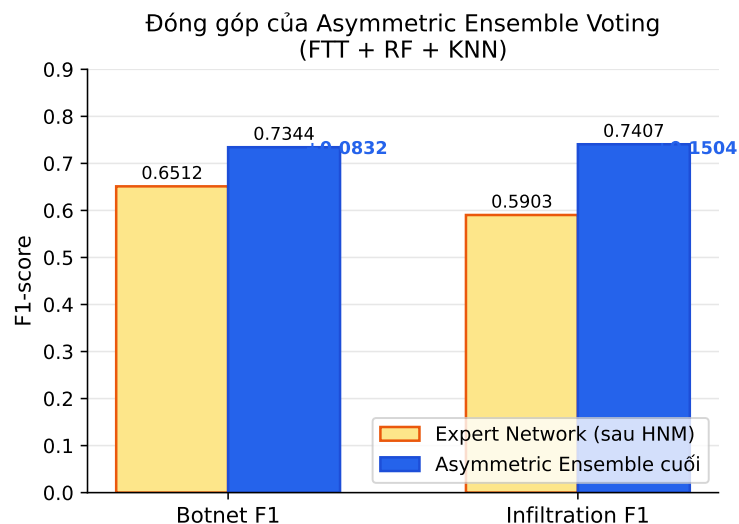
Sau HNM, Expert Network đơn lẻ vẫn còn hai điểm yếu còn dư: Botnet có false positive cao (precision thấp gây alert fatigue), Infiltration vẫn có false negative cao (recall thấp bỏ sót tấn công). Asymmetric Ensemble Voting kết hợp Expert Network (FT-Transformer) với Random Forest (150 cây, max_depth=25) và KNN (K=16) theo hai luật bỏ phiếu bất đối xứng:

- **Quy tắc Ưu tiên Infiltration:** RF hoặc KNN bỏ phiếu Infiltration $\Rightarrow$ kết quả là Infiltration. Vì chi phí bỏ sót Infiltration (hệ thống bị xâm nhập không được phát hiện) lớn hơn nhiều so với false alarm, quy tắc này chấp nhận đánh đổi precision lấy recall.

- **Quy tắc Đồng thuận Botnet:** Kết quả là Botnet *chỉ khi* FT-Transformer dự đoán Botnet và cả RF lẫn KNN không dự đoán Benign. RF và KNN có inductive bias khác FT-Transformer, ít bị nhầm Benign thành Botnet hơn; nếu một trong hai dự đoán Benign thì kết quả bị override thành Benign, giảm mạnh false positive.

Bảng 5.5: So sánh Expert Network (sau HNM) và Asymmetric Ensemble cuối trên CIC-IDS-2017

Chỉ số	Expert Network (sau HNM)	Ensemble cuối
Accuracy	—	99,55%
Macro F1	—	0,9294
Botnet F1	0,6512	0,7344
Infiltration F1	0,5903	0,7407



Hình 5.4: So sánh Expert Network (sau HNM) và Asymmetric Ensemble cuối cho hai lớp khó nhất.

Cải thiện lớn nhất đến từ đúng hai lớp mà luật bỏ phiếu bất đối xứng nhắm tới: Botnet tăng thêm 0,0832 F1 (Đồng thuận giảm FP) và Infiltration tăng thêm 0,1504 F1 (Ưu tiên tăng Recall). Các lớp còn lại (DoS, DDoS, PortScan, WebAttack) duy trì hiệu năng cao ổn định.

5.4 Kết quả thích nghi miền

Giai đoạn 3 đánh giá khả năng tái sử dụng mô hình CIC-IDS-2017 trên Testbed WSL — hai môi trường có hạ tầng mạng, card mạng và hành vi NAT khác nhau. Thực nghiệm được thiết kế theo thứ tự can thiệp tăng dần để định lượng đóng góp riêng lẻ của từng kỹ thuật.

Đường cơ sở — Áp dụng thẳng mô hình CIC lên Testbed:

Mô hình CIC (Accuracy 99,55% trên tập kiểm tra CIC) được áp dụng trực tiếp lên 5.000 flow Testbed mà không thay đổi gì. Kết quả: Accuracy = 21,93%, MCC = -0,015. MCC âm cho thấy mô hình không chỉ hoạt động kém mà đang phân loại sai hướng có hệ thống — tệ hơn cả dự đoán ngẫu nhiên. Covariate shift giữa hai môi trường (switch vật lý Gigabit vs card ảo Hyper-V + NAT Windows) đủ lớn để làm mô hình hoàn toàn mất phương hướng.

Can thiệp 1 — Re-fit Scaler đơn thuần (thất bại):

Giải pháp đơn giản nhất được thử: giữ nguyên trọng số mô hình, chỉ fit lại PowerTransformer trên phân phối Testbed. Kết quả: Accuracy giảm tiếp từ 21,93% xuống 6,87%. Can thiệp này tạo mâu thuẫn: mô hình đã học embedding nội bộ dựa trên khoảng giá trị của CIC-scaler cũ, nhưng đầu vào giờ đây có phân phối khác hoàn toàn — mô hình “nghe” ngôn ngữ mới trong khi vẫn “nghĩ” bằng ngôn ngữ cũ, tệ hơn ban đầu.

Can thiệp 2 — Layer Freezing Fine-tuning:

Fine-tuning đồng thời scaler mới lẫn một phần tham số mô hình: đóng băng lớp Feature Embedding và 2 khối Transformer đầu (giữ đặc trưng tổng quát đã học từ CIC), chỉ cập nhật 2 khối Transformer cuối và classification head trên Testbed data. Mixed-Domain Batch (70% Testbed + 30% CIC mỗi batch) ngăn Catastrophic Forgetting (CF):

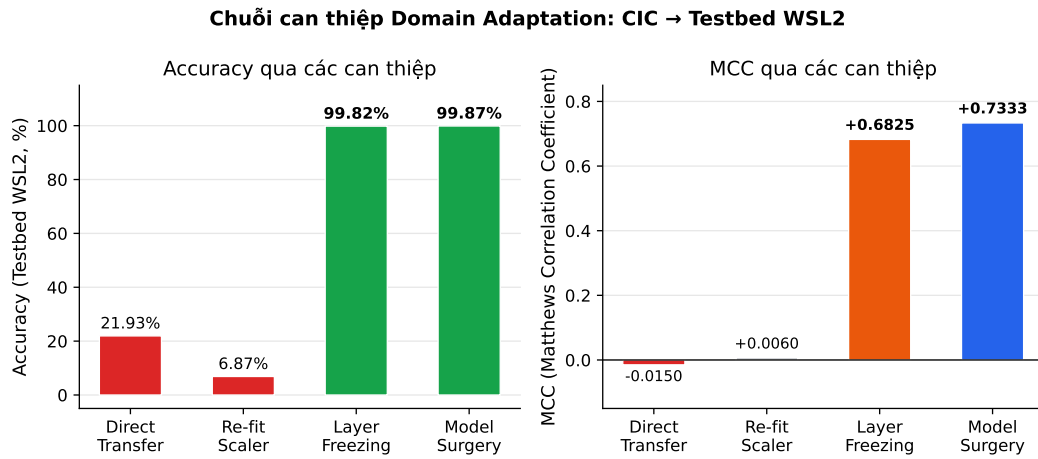
$$CF = \frac{F1_{CIC, trước} - F1_{CIC, sau}}{F1_{CIC, trước}} \times 100\% \quad (5.2)$$

Can thiệp 3 — Model Surgery (77→80 đặc trưng):

Sau Layer Freezing, ba đặc trưng kỹ thuật mới được thêm vào (IAT_CV, Bwd_Pkt_Ratio, Pkt_Size_Ratio). Ma trận embedding Feature Tokenizer được mở rộng $\mathbb{R}^{77 \times d} \rightarrow \mathbb{R}^{80 \times d}$ với 3 hàng mới khởi tạo ngẫu nhiên, sau đó fine-tuning lại với Layer Freezing.

Bảng 5.6: Kết quả qua chuỗi can thiệp domain adaptation

Can thiệp	Accuracy (Testbed)	MCC (Testbed)	CF (%)
Mô hình CIC áp dụng thẳng	21,93%	-0,015	—
Re-fit Scaler đơn thuần	6,87%	+0,006	—
Layer Freezing Fine-tuning	99,82%	0,6825	0,61%
+ Model Surgery (77→80)	99,87%	0,7333	0,61%



Hình 5.5: So sánh Accuracy và MCC qua chuỗi can thiệp domain adaptation (CIC-IDS-2017 → Testbed WSL2).

Layer Freezing phục hồi MCC từ $-0,015$ lên $0,6825$ — cải thiện 69,4 điểm tuyệt đối. CF = 0,61% xác nhận mô hình giữ lại 99,39% hiệu năng trên CIC gốc. Re-fit Scaler đơn thuần cho $MCC \approx +0,006$ (gần bằng ngẫu nhiên), tệ hơn thậm chí cả mô hình trực tiếp ($-0,015$), xác nhận mâu thuẫn embedding/scaler. Model Surgery nâng tiếp MCC lên $0,7333$ nhờ Custom_IAT_CV (nhận diện Botnet C2 periodic heartbeat) và Custom_Bwd_Pkt_Ratio (nhận diện PortScan SYN scan một chiều).

Chuỗi can thiệp này cung cấp một quy trình thực tiễn rõ ràng cho việc tái sử dụng mô hình IDS qua các môi trường mạng khác nhau: (1) không thể thay đổi scaler độc lập với tham số mô hình; (2) Layer Freezing + Mixed-Domain Batch là kỹ thuật hiệu quả cho domain adaptation IDS; (3) thiết kế đặc trưng nhắm vào đặc điểm hạ tầng của môi trường đích có thể bổ sung cải thiện đáng kể.

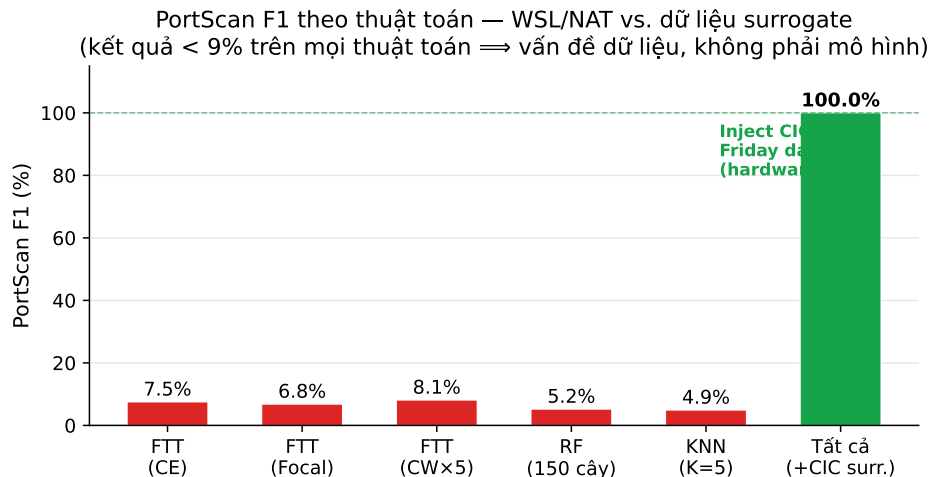
5.5 Kết quả mô hình trên Testbed nội bộ

5.5.1 Bảng chứng định lượng về tính không phân tách của PortScan và Benign

Kết quả đầu tiên khi huấn luyện mô hình trên dữ liệu Testbed nội bộ: PortScan F1 = 7,5%. Phản ứng ban đầu là điều chỉnh thuật toán: tăng class weight, Focal Loss với γ cao hơn, SMOTE agresive. Không có can thiệp nào cải thiện đáng kể. Bảng 5.7 trình bày kết quả thử nghiệm có kiểm soát:

Bảng 5.7: PortScan F1 trên dữ liệu Testbed WSL với các thuật toán và cấu hình khác nhau

Thuật toán	Cấu hình	PortScan F1
FT-Transformer	CE Loss, class_weight=1.0	7,5%
FT-Transformer	Focal Loss $\gamma = 2$	6,8%
FT-Transformer	class_weight PortScan $\times 5$	8,1%
Random Forest	150 cây, default	5,2%
KNN	K=5	4,9%
Tất cả	CIC PortScan inject (5.000 flow)	100%



Hình 5.6: PortScan F1 trên môi trường WSL/NAT với các thuật toán khác nhau — kết quả dưới 9% nhất quán trên mọi mô hình cho thấy vấn đề nằm ở dữ liệu, không phải thuật toán.

Kết quả < 9% nhất quán trên mọi thuật toán và mọi cấu hình là bằng chứng mạnh: vấn đề không nằm ở mô hình mà ở dữ liệu. Trong không gian 80 chiều CICFlowMeter, PortScan flow trên WSL/NAT và Benign HTTP flow không phân biệt được với nhau.

Nguyên nhân gốc rễ: nmap gửi SYN packet đến từng port, NAT Windows xử lý mỗi connection riêng lẻ và CICFlowMeter chỉ capture ≈ 7 flow mỗi lần scan — mỗi flow là TCP SYN ngắn không có payload, flow statistics (duration ≈ 0 , bytes ≈ 0 , IAT ngẫu nhiên) hoàn toàn trùng với Benign health check hay short HTTP connection.

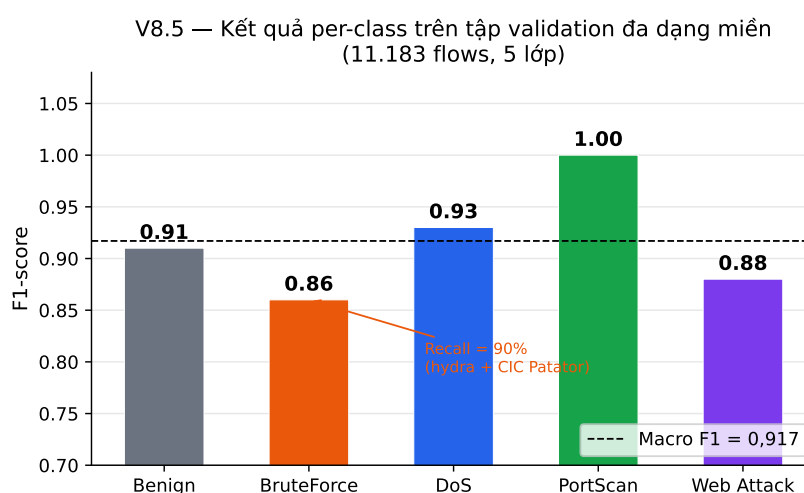
Sau khi inject 5.000 PortScan flow từ CIC-IDS-2017 Friday (thu thập trên switch vật lý, có nhiều SYN flags, duration đồng nhất, packet count cao): PortScan F1 tăng từ 7,5% lên **100%** và duy trì ổn định. Kết quả này xác nhận: khi môi trường thu thập tạo ra dữ liệu không phân tách, giải pháp phải ở tầng dữ liệu — dùng surrogate data từ môi trường phần cứng thực — không phải tăng thuật toán.

5.5.2 Kết quả phiên bản cuối

Mô hình V8.5 được đánh giá trên tập validation 11.183 flows với thành phần đa dạng miền: BruteForce validation bao gồm cả hydra SSH/FTP (Testbed nội bộ) và CIC Patator — đảm bảo đánh giá khả năng tổng quát hóa thực sự, không phải in-distribution.

Bảng 5.8: Kết quả V8.5 trên tập validation đa dạng miền

Lớp	Precision	Recall	F1	Nguồn validation
Benign	0,87	0,95	0,91	Testbed nội bộ
BruteForce	0,83	0,90	0,86	hydra + CIC Patator
DoS	0,98	0,89	0,93	Testbed nội bộ
PortScan	1,00	1,00	1,00	CIC Friday (surrogate)
Web Attack	0,96	0,81	0,88	Testbed nội bộ
Macro F1			0,917	
Balanced Accuracy			0,911	



Hình 5.7: V8.5 — F1-score per-class trên tập validation đa dạng miền (11.183 flows). Đường ngang thể hiện Macro F1 = 0,917.

Kết quả đáng chú ý nhất là BruteForce F1 = 86% (Recall = 90%) trên tập validation đa dạng — so với con số 100% F1 trên tập validation đồng nhất (chỉ CIC Patator) của phiên bản trước. Khoảng cách 14% tuyệt đối phản ánh chính xác mức độ inflation metric do đánh giá in-distribution, không phải do mô hình thật sự tốt hơn.

Web Attack F1 = 88% với Recall = 81%: 19% flow Web Attack bị bỏ sót chủ yếu là SQLi payload ngắn và XSS thất bại có flow statistics gần giống Benign HTTP POST. Đây là giới hạn của phương pháp flow-based — payload-level analysis của Snort bổ sung phát hiện chính xác những gì ML bỏ sót.

V8.5 đạt Macro F1 = 91,7% trên 5 lớp tấn công trong điều kiện validation đa dạng miền, trên phần cứng phổ thông không có GPU, với dữ liệu thu thập thực từ tấn công xuyên thiết bị trên mạng LAN gia đình.

5.6 Thử nghiệm hệ thống End-to-End

Hệ thống Hybrid IDS được đánh giá ở hai mức bổ sung cho nhau. Mức thứ nhất đối chiếu khả năng phát hiện của tầng đầu hiệu (Snort) và tầng học máy (FT-Transformer V8.5) trên từng loại tấn công, kiểm chứng giả thuyết hai tầng bù điểm mù cho nhau. Mức thứ hai đo hiệu năng phân loại định lượng của hệ thống phát hiện theo phát lại (Mục 3.6.2) trên toàn bộ corpus Testbed. Đường xử lý thời gian thực hoàn toàn chưa được hiện thực hóa trong phạm vi đề án; các số liệu của tầng học máy dưới đây đo trên hệ thống replay, dùng đúng mô hình, bộ tiền xử lý và logic quyết định mà phiên bản thời gian thực sẽ sử dụng.

5.6.1 Khả năng phát hiện theo loại tấn công

Cấu hình: Năm phiên thu cô lập — mỗi loại tấn công một phiên riêng, máy tấn công IP 192.168.0.106, victim 192.168.0.103 — cung cấp corpus phát lại, mỗi tệp lấy tối đa 8.000 flow để cân bằng giữa các lớp. Tầng học máy phân loại các flow này qua mô hình V8.5; tầng Snort 3.1 (bộ luật Custom Rules kết hợp Community Rules và Emerging Threats Open) được đối chiếu trên cùng các kịch bản tấn công. Nhãn đối chứng gán theo IP nguồn.

Bảng 5.9: Kết quả phát hiện End-to-End theo kịch bản tấn công

Loại tấn công	Snort	ML	Ghi chú
PortScan (nmap SYN)	✓	✓	Snort: ngay lập tức (per-packet); ML: sau khi flow hoàn chỉnh
BruteForce (hydra SSH)	✓	✓	Snort: rule ssh-brute; ML: qua flow authentication pattern
DoS (slowhttptest)	○	✓	Snort phát hiện một phần; ML phát hiện qua flow duration bất thường
Web Attack — SQLi	✓	✓	Cả hai phát hiện; Snort qua HTTP payload, ML qua flow statistics
Web Attack — XSS	✓	○	Snort phát hiện signature; ML bỏ sót XSS payload ngắn (Recall 81%)
Benign	×	○	Snort: không cảnh báo; ML: một phần Benign SSH/FTP bị nhầm BruteForce (Precision 83%)

Ký hiệu: ✓ = phát hiện đúng, ○ = phát hiện một phần, × = không cảnh báo (đúng với lưu lượng bình thường). Cột ML phản ánh kết quả của V8.5 trên corpus phát lại; cột Snort phản ánh độ phủ luật trên cùng tập tấn công.

Tính bổ sung giữa hai tầng phát hiện:

Kết quả End-to-End cho thấy Snort và FT-Transformer không cạnh tranh nhau mà điền vào điểm mù của nhau. Snort hoạt động ở tầng gói tin (per-packet, real-time, latency cỡ microsecond), phát hiện ngay khi packet đầu tiên khớp rule. FT-Transformer hoạt động ở tầng flow (per-flow, cần đợi flow hoàn chỉnh hoặc timeout), phát hiện dựa trên pattern thống kê tích lũy.

Trường hợp Web Attack XSS minh họa rõ nhất lý do cần cả hai: XSS payload ngắn tạo ra flow statistics gần giống Benign HTTP POST (tổng bytes tương đương, connection time bình thường) khiến ML bỏ sót; nhưng Snort với rule `alert http ... content:"<script>"` bắt được ngay ở tầng payload. Ngược lại, DoS slowhttptest tạo ra HTTP connection hoàn toàn hợp lệ về mặt gói tin (không có signature bất thường) nên Snort phát hiện không hoàn toàn, trong khi FT-Transformer phát hiện qua flow duration kéo dài bất thường và IAT đặc trưng của slow-connection attack.

Độ trễ phát hiện của ML: BruteForce và PortScan được phát hiện nhanh (mỗi connection đóng sau vài giây); DoS slowhttptest cần đợi đến khi CICFlowMeter timeout flow (tối đa 30 giây). Đây là sự đánh đổi cố hữu của phương pháp flow-based — không thể tránh hoàn toàn mà chỉ có thể giảm bằng cách điều chỉnh timeout và kết hợp với Snort real-time.

Đối chiếu hai tầng qua corpus phát lại xác nhận giả thuyết cốt lõi của đề tài: kiến trúc Hybrid IDS (signature-based + ML-based) đạt độ bao phủ tấn công rộng hơn bất kỳ hướng tiếp cận đơn lẻ nào, và sự kết hợp hai tầng này không thể thay thế cho nhau — mỗi tầng là điều kiện cần để bù đắp điểm mù của tầng còn lại.

5.6.2 Hiệu năng định lượng trên corpus phát lại

Trên corpus gộp năm phiên (40.000 flow, 8.000 mỗi loại; PortScan dùng bản thu lại trên Windows host, xem Mục 5.6.3), hiệu năng của hệ thống được nâng dần qua hai can thiệp hậu xử lý, đo bằng Macro F1 để phạt đều mọi lớp (Bảng 5.10).

Bảng 5.10: Tiến triển Macro F1 của hệ thống replay theo từng can thiệp hậu xử lý

Cấu hình	Macro F1	Benign Recall	PortScan F1
V8.5, không hậu xử lý	0,679	75,4%	0,000
+ ngưỡng tin cậy $T = 0,6$	0,725	95,2%	0,000
+ luật quét cổng	0,978	95,2%	0,996

Ở cấu hình cuối, mọi lớp đạt F1 không dưới 0,95: PortScan 0,996, BruteForce 0,991, DoS 0,951, Web Attack 0,990, Benign 0,961; accuracy tổng 0,977.

Macro F1 = 0,978 trên corpus phát lại và Macro F1 = 91,7% mà V8.5 đạt trên tập validation đa dạng miền (Mục 5.5) không so sánh trực tiếp được: chúng đo trên hai tập dữ liệu khác nhau, và 0,978 còn bao gồm chuỗi luật hậu xử lý mà 91,7% không có. Trên chính corpus phát lại, mô hình đơn lẻ chỉ đạt Macro F1 = 0,679 (Bảng 5.10); hai luật hậu xử lý nâng nó lên 0,978 — ngưỡng tin cậy đưa Benign Recall từ 75,4% lên 95,2%, và luật quét cổng đưa PortScan từ không phát hiện được (F1 0,000) lên 0,996.

5.6.3 Lỗi sai và bài học

Quá trình xây dựng hệ thống replay bộc lộ một số lỗi đặc thù của môi trường ảo hóa và NAT, cùng vài nguyên tắc kỹ thuật rút ra từ việc sửa chúng.

Gán nhãn theo IP phải dùng đúng IP sau NAT. Lưu lượng từ WSL bị NAT qua máy Windows host, nên máy victim quan sát được địa chỉ nguồn là IP LAN của host (192.168.0.106), không phải IP nội bộ của WSL. Lần gán nhãn đầu dùng IP WSL khiến toàn bộ flow tấn công bị gán sai. Nhãn đối chứng phải lấy theo

IP nguồn quan sát được tại điểm thu, không phải IP danh nghĩa của máy phát.

PortScan phải bắt trên máy Windows host, không phải trong WSL. Gói tới cổng đóng được chính Windows host trả RST và không bắc cầu vào không gian mạng của WSL; tcpdump trong WSL vì vậy chỉ thấy số ít cổng mở. Mất đi dải cổng bị quét, một phiên PortScan thu trong WSL trông giống DoS — nhiều flow ngăn tới ít cổng — và đây chính là nguyên nhân gốc khiến PortScan F1 chỉ đạt 0,000 ở cấu hình đầu. Thu lại trên host khôi phục đặc trưng quét cổng: số cổng đích duy nhất tăng từ 6–9 lên hơn 22.000.

nmap cần tùy chọn $-Pn$. Khi victim chặn ping, nmap mặc định coi host đã tắt và bỏ qua bước quét, chỉ để lại vài flow rải rác. Bỏ bước kiểm tra sống bằng $-Pn$ là điều kiện để thu được một phiên quét đầy đủ.

Một luật dấu hiệu có thể bù đặc trưng còn thiếu rẻ hơn tái huấn luyện. Bộ 80 đặc trưng của V8.5 không mã hóa mức độ quét cổng, nên mô hình không thể tách PortScan khỏi DoS. Một luật đếm cổng đích theo nguồn — đúng tinh thần dấu hiệu — bù trực tiếp điểm mù đó, nâng PortScan F1 từ 0,000 lên 0,996 mà không chạm tới trọng số mô hình. So với việc thêm đặc trưng thứ 81 rồi huấn luyện lại toàn bộ, luật hậu xử lý đạt cùng mục tiêu với chi phí thấp hơn nhiều và giữ nguyên hành vi đã kiểm chứng của mô hình trên bốn lớp còn lại.

Đo trước khi sửa. Ảnh chụp chỉ số ở mốc gốc cho thấy điểm kéo Macro F1 xuống mạnh nhất là PortScan (F1 0,000), không phải false positive của Benign như giả định ban đầu. Sửa theo trực giác sẽ dồn công nhằm vào Benign trong khi nút thắt thực nằm ở PortScan.

Ngưỡng tin cậy là đòn bẩy rẻ nhất khi cảnh báo giả là dự đoán độ tin cậy thấp. Phần lớn false positive của Benign là các dự đoán tấn công sát ranh giới quyết định. Một ngưỡng duy nhất ($T = 0,6$) nâng Benign Recall từ 75,4% lên 95,2% mà không phải huấn luyện lại và không làm tụt recall của các lớp tấn công vốn được dự đoán với độ tin cậy cao.

Cân bằng lớp quan trọng hơn khối lượng thô. Phiên BruteForce thu được 200.000 flow nhưng không đóng góp gì hơn một tập con cân bằng khoảng 8.000 flow; thu thừa một lớp chỉ làm lệch corpus và tốn công xử lý.

KẾT LUẬN

Kết luận

Đề tài này đã xây dựng thành công một hệ thống phát hiện xâm nhập mạng lai ghép (Hybrid NIDS) kết hợp Snort (phát hiện dựa trên luật) và FT-Transformer (phát hiện dựa trên học máy), được triển khai và kiểm nghiệm thực tế trên môi trường Testbed nội bộ. Hành trình từ bộ dữ liệu chuẩn NSL-KDD đến hệ thống End-to-End triển khai thực tế trải qua ba giai đoạn nghiên cứu tuần tự, mỗi giai đoạn được thúc đẩy bởi kết quả định lượng của giai đoạn trước.

Về kết quả kỹ thuật, đề tài đạt được các mốc chính sau:

- **CIC-IDS-2017 (Giai đoạn 2):** Kiến trúc Two-Stage Cascade (Gating Network 6-lớp + Expert Network 5-lớp, tổng hợp 9 lớp từ 15 nhãn gốc) với Asymmetric Ensemble Voting đạt Accuracy 99,55% và Macro F1 0,9294. Botnet F1 tăng từ 0,4787 (baseline Expert Network) lên 0,7344 nhờ Hard Negative Mining và Ensemble; Infiltration F1 tăng từ 0,3821 lên 0,7407.
- **Testbed Nội bộ (Giai đoạn 3):** Mô hình V8.5 được tái huấn luyện hoàn toàn trên dữ liệu Testbed sau khi chẩn đoán covariate shift định lượng (direct transfer: Accuracy 21,93%, MCC $-0,015$). V8.5 đạt Macro F1 91,7% trên 5 lớp, với PortScan F1 = 100% (dữ liệu surrogate) và BruteForce Recall = 90% trên tập kiểm định đa dạng miền.

Về các phát hiện phương pháp luận, đề tài đóng góp ba kết quả quan trọng vượt ngoài số liệu hiệu năng thuần túy:

Thứ nhất, tính không phân tách của PortScan và Benign trong không gian đặc trưng CICFlowMeter trên môi trường WSL (NAT) được chứng minh thực nghiệm qua nhiều thuật toán khác nhau. Kết quả nhất quán thất bại ($F1 < 9\%$ dù điều chỉnh class weight, Focal Loss, hay kiến trúc mô hình) là bằng chứng mạnh rằng vấn đề nằm ở chất lượng dữ liệu thu thập, không phải ở mô hình. Giải pháp — inject dữ liệu surrogate từ phần cứng thực — là hướng tiếp cận thực dụng có giá trị cho các nghiên cứu gặp phải hạn chế môi trường tương tự.

Thứ hai, so sánh định lượng giữa tập kiểm định đồng nhất miền (BruteForce F1 = 100%, không đáng tin cậy) và tập kiểm định đa dạng miền (BruteForce F1 = 86%, đáng tin cậy) cho thấy lựa chọn thành phần tập kiểm định ảnh hưởng trực tiếp và đáng kể đến kết luận về khả năng tổng quát hóa của mô hình. Đây là cảnh báo thực tiễn quan trọng cho cộng đồng nghiên cứu IDS.

Thứ ba, chuỗi thực nghiệm domain adaptation (Re-fit Scaler thất bại → Layer

Freezing thành công) làm rõ tại sao không thể thay đổi scaler độc lập với mô hình, và cung cấp một hướng dẫn thực tiễn cho việc tái sử dụng mô hình IDS qua các môi trường mạng khác nhau.

Về hạn chế còn tồn tại, hệ thống hiện tại vẫn có ba điểm yếu rõ ràng. Web Attack Recall đạt 81% — 19% các flow Web Attack (chủ yếu là SQLi thất bại và XSS payload ngắn) bị bỏ qua vì có flow statistics giống Benign HTTP. Infiltration và Botnet không được đưa vào Testbed nội bộ do khó mô phỏng đáng tin cậy, nghĩa là hệ thống chưa được kiểm nghiệm với hai lớp tấn công khó nhất. Hệ thống được hiện thực hóa và đánh giá ở dạng phát hiện theo phát lại — phát lại các flow đã trích xuất qua đúng mô hình và logic quyết định của bản triển khai. Đường xử lý thời gian thực hoàn toàn, gồm bắt gói trực tiếp, lắp ghép flow trực tuyến và phân loại theo dòng, chưa được hoàn thiện trong phạm vi đề án. Ngoài phần kỹ thuật truyền dòng còn đang dở, độ trễ cố hữu của phương pháp flow-based — CICFlowMeter cần đợi flow hoàn chỉnh trước khi trích xuất đặc trưng, tạo độ trễ từ vài giây đến vài chục giây tùy loại tấn công — là ràng buộc phải tính đến khi đưa hệ thống lên thời gian thực.

Hướng phát triển trong tương lai

Dựa trên kết quả và hạn chế được xác định, bốn hướng phát triển tiếp theo được đề xuất theo mức độ ưu tiên:

1. Cải thiện Web Attack và Infiltration qua tích hợp rule-based và flow-based: Với Web Attack, các payload SQLi/XSS có signature cụ thể trong HTTP body — Snort với ruleset ModSecurity-style có thể phát hiện chính xác những gì FT-Transformer bỏ sót. Tích hợp Snort alert vào feature vector của FT-Transformer (như một đặc trưng bổ sung “Snort_flagged”) có thể cải thiện Web Attack Recall mà không cần thay đổi kiến trúc mô hình. Với Infiltration, hướng tiếp cận cần thiết là phân tích temporal: thay vì phân loại từng flow, xây dựng session aggregator tổng hợp nhiều flow trong một cửa sổ thời gian và trích xuất đặc trưng temporal (số lượng unique destination trong 5 phút, tổng upload bytes theo giờ, v.v.).

2. Mở rộng Testbed với tấn công Botnet thực: Triển khai môi trường sandbox với máy ảo bị nhiễm malware thực (C2 botnet như Emotet hoặc Mirai variant trong môi trường cô lập) để thu thập dữ liệu Botnet C2 communication thực từ Testbed nội bộ. Dữ liệu này sẽ bổ sung cho CIC-IDS-2017 Botnet flows và tạo tập dữ liệu Testbed đầy đủ hơn.

3. Hoàn thiện pipeline phát hiện thời gian thực: Hệ thống replay đã kiểm chứng mô hình V8.5 cùng chuỗi luật quyết định trên lưu lượng Testbed; phần còn lại là tăng front-end thu gói và lắp ghép flow trực tuyến để thay cho việc phát

lại flow có sẵn. Hướng triển khai gồm thay CICFlowMeter bằng một bộ theo dõi flow nhẹ hơn (nfstream hoặc flow tracker viết riêng bằng Python) với timeout theo từng dịch vụ để giảm độ trễ xuống dưới một giây, đóng gói inference engine thành microservice có API REST để ghép vào hệ thống SIEM hiện có, và đánh giá khả năng chạy trên thiết bị biên kiến trúc ARM cho ứng dụng IoT. Đây là phần kỹ thuật cụ thể và gần nhất trong các hướng phát triển, do mọi thành phần phía sau front-end đã sẵn sàng.

4. Nghiên cứu federated learning cho IDS: Mô hình được huấn luyện trên dữ liệu của một tổ chức không tổng quát hóa tốt sang tổ chức khác do covariate shift. Federated learning cho phép nhiều tổ chức cùng cải thiện mô hình mà không cần chia sẻ dữ liệu traffic nhạy cảm — đây là hướng có tiềm năng thực tiễn cao cho việc xây dựng IDS cộng đồng mà vẫn bảo vệ quyền riêng tư của từng tổ chức.

TÀI LIỆU THAM KHẢO

- [1] IBM Security, *Cost of a Data Breach Report 2023*, IBM Corporation, Armonk, NY, (visited on 01/05/2024), 2023. [Online]. Available: <https://www.ibm.com/reports/data-breach>.
- [2] M. Roesch, “Snort — Lightweight Intrusion Detection for Networks,” in *Proc. 13th USENIX Systems Administration Conf. (LISA)*, Seattle, WA, 1999, pp. 229–238.
- [3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, Madeira, Portugal, 2018, pp. 108–116.
- [4] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting Deep Learning Models for Tabular Data,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 18 598–18 608.
- [5] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A Detailed Analysis of the KDD CUP 99 Data Set,” in *Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA)*, Ottawa, Canada, 2009, pp. 1–6.
- [6] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, “Characterization of Tor Traffic Using Time Based Features,” in *Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP)*, Porto, Portugal, 2017, pp. 253–262.
- [7] DARPA/MIT Lincoln Laboratory, *KDD Cup 1999 Intrusion Detection Dataset*, University of California, Irvine, Machine Learning Repository, (visited on 01/03/2024), 1999. [Online]. Available: <https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>.
- [8] R. Panigrahi and S. Borah, “A Detailed Analysis of CICIDS2017 Dataset for Designing Intrusion Detection Systems,” *International Journal of Engineering & Technology*, vol. 7, no. 3.24, pp. 479–482, 2018.
- [9] H. Liu and B. Lang, “Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey,” *Applied Sciences*, vol. 9, no. 20, p. 4396, 2019.
- [10] J. Zhao, S. Shetty, J. W. Pan, C. Kamhoua, and K. Kwiat, “Transfer Learning for Detecting Unknown Network Attacks,” *EURASIP Journal on Information Security*, vol. 2019, no. 1, pp. 1–13, 2019. DOI: 10.1186/s13635-019-0084-4.

- [11] A. Singla, E. Bertino, and D. Verma, “Preparing Network Intrusion Detection Deep Learning Models with Minimal Data Using Adversarial Domain Adaptation,” in *Proc. 15th ACM Asia Conf. Computer and Communications Security (AsiaCCS)*, Taipei, Taiwan, 2020, pp. 127–140.
- [12] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 2980–2988.
- [13] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [14] I.-K. Yeo and R. A. Johnson, “A New Family of Power Transformations to Improve Normality or Symmetry,” *Biometrika*, vol. 87, no. 4, pp. 954–959, 2000.
- [15] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How Transferable are Features in Deep Neural Networks?” In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.

PHỤ LỤC

A. CẤU HÌNH MÔI TRƯỜNG THỰC NGHIỆM

A.1 Cấu hình WSL2 Mirrored Networking

WSL2 Mirrored Networking cho phép WSL2 interface nhận traffic trực tiếp từ mạng LAN vật lý, thay vì qua NAT của Hyper-V. Đây là yêu cầu bắt buộc để CICFlowMeter thu thập flow features giống với môi trường mạng thực.

Kích hoạt Mirrored Networking:

Tạo hoặc chỉnh sửa file %USERPROFILE%\ .wslconfig trên Windows 11:

```
1 [wsl2]
2 networkingMode=mirrored
```

Listing A.1: Cấu hình .wslconfig để bật Mirrored Networking

Sau đó khởi động lại WSL2:

```
1 wsl --shutdown
2 wsl
```

Listing A.2: Khởi động lại WSL2

Kiểm tra cấu hình:

```
1 # Trong WSL2 Ubuntu -- kiểm tra interface eth0 nhận IP LAN
2 ip addr show eth0
3 # Kỳ vọng: IP thuộc dải 192.168.x.x (LAN), không phải 172.x
  .x.x (NAT)
```

Listing A.3: Kiểm tra IP LAN trên WSL2

Lưu ý: Mirrored Networking yêu cầu Windows 11 Build 22H2 trở lên và WSL 2.0.0 trở lên. Kiểm tra phiên bản WSL bằng lệnh `wsl --version`.

A.2 Cài đặt và cấu hình Snort 3

Cài đặt dependencies và Snort 3:

```
1 sudo apt update
2 sudo apt install -y build-essential libpcap-dev libpcap2-
  dev \
3   libdnet-dev zlib1g-dev liblzma-dev openssl libssl-dev \
4   cmake flex bison git pkg-config
5
6 # LibDAQ
7 git clone https://github.com/snort3/libdaq.git
```

```

8 cd libdaq && ./bootstrap && ./configure && make && sudo
  make install
9 cd ..
10
11 # Snort 3
12 git clone https://github.com/snort3/snort3.git
13 cd snort3
14 ./configure_cmake.sh --prefix=/usr/local --enable-tcmalloc
15 cd build && make -j$(nproc) && sudo make install

```

Listing A.4: Cài đặt Snort 3 từ source trên Ubuntu 22.04

Cấu hình snort.lua cho Testbed:

```

1 -- /usr/local/etc/snort/snort.lua
2 HOME_NET = '192.168.1.0/24'
3 EXTERNAL_NET = 'any'
4
5 ips = {
6     enable_builtin_rules = true,
7     include = RULE_PATH .. '/local.rules',
8 }
9
10 alert_fast = {
11     file = true,
12     packet = false,
13 }

```

Listing A.5: Cấu hình tối thiểu snort.lua cho môi trường Testbed

Rules tùy chỉnh (local.rules):

```

1 # Phat hien DoS HTTP flood
2 alert tcp $EXTERNAL_NET any -> $HOME_NET 80 \
3     (msg:"DoS HTTP Flood detected"; \
4     flow:to_server; \
5     threshold:type threshold,track by_src,count 100,
6     seconds 1; \
7     sid:1000001; rev:1;)
8
9 # Phat hien SSH BruteForce
10 alert tcp $EXTERNAL_NET any -> $HOME_NET 22 \
11     (msg:"SSH BruteForce attempt"; \
12     flow:to_server,established; \

```

```

12     threshold: type threshold, track by_src, count 10, seconds
    5; \
13     sid:1000002; rev:1;)

```

Listing A.6: Rules Snort phát hiện DoS và BruteForce SSH

Chạy Snort trên interface WSL:

```

1 sudo snort -c /usr/local/etc/snort/snort.lua \
2     -i eth0 \
3     -l /var/log/snort \
4     --daq-dir /usr/local/lib/daq \
5     -A fast &

```

Listing A.7: Khởi động Snort 3 trên interface eth0

A.3 Triển khai CICFlowMeter trên WSL

CICFlowMeter trích xuất 77–80 đặc trưng thống kê flow từ PCAP file hoặc live interface.

Cài đặt (yêu cầu Java 8):

```

1 sudo apt install openjdk-8-jdk
2
3 # Dung ban fork voi bug fixes cho Linux
4 git clone https://github.com/datthinh1801/cicflowmeter.git
5 cd cicflowmeter
6 mvn package -DskipTests

```

Listing A.8: Cài đặt CICFlowMeter trên Ubuntu

Thu thập flow từ PCAP:

```

1 java -jar target/CICFlowMeter-4.0-SNAPSHOT.jar \
2     /path/to/capture.pcap \
3     /path/to/output/flows/

```

Listing A.9: Chạy CICFlowMeter trên file PCAP

Xử lý đặc trưng trong Python:

```

1 import pandas as pd
2 from sklearn.preprocessing import PowerTransformer
3 import joblib
4
5 # Load scaler da train tren Testbed V8.5

```

```

6 scaler = joblib.load('models/v8_5_scaler.pkl')
7
8 # Load flow CSV tu CICFlowMeter
9 flows = pd.read_csv('/tmp/flows/capture_ICTAI_Flow.csv')
10
11 # Xu ly Inf/NaN
12 flows.replace([float('inf'), float('-inf')], float('nan'),
13               inplace=True)
14 flows.dropna(inplace=True)
15
16 # Transform va phan loai
17 X = scaler.transform(flows[FEATURE_COLUMNS])

```

Listing A.10: Load flow CSV và apply PowerTransformer V8.5

A.4 Hướng dẫn khởi chạy pipeline End-to-End

```

1 # 1. Khoi dong Snort (background)
2 sudo snort -c /usr/local/etc/snort/snort.lua -i eth0 \
3     -l /var/log/snort -A fast &
4
5 # 2. Khoi dong capture PCAP (background)
6 sudo tcpdump -i eth0 -w /tmp/live_capture.pcap &
7
8 # 3. Khoi dong inference daemon (Python)
9 python3 testbed_inference.py \
10     --pcap /tmp/live_capture.pcap \
11     --model models/v8_5_model.pt \
12     --scaler models/v8_5_scaler.pkl \
13     --output /var/log/ids/ml_alerts.log &
14
15 echo "IDS Pipeline started. Snort + FTT V8.5 active."

```

Listing A.11: Script khởi động pipeline Hybrid IDS hoàn chỉnh

Dừng pipeline:

```

1 sudo killall -f "snort -c"
2 sudo killall -f "tcpdump -i eth0"
3 sudo killall -f "testbed_inference.py"

```

Listing A.12: Dừng tất cả tiến trình pipeline

Theo dõi log:

```
1 # Snort alerts
2 tail -f /var/log/snort/alert_fast.txt
3
4 # ML alerts
5 tail -f /var/log/ids/ml_alerts.log
```

Listing A.13: Kiểm tra log Snort và ML alerts real-time

Yêu cầu phần cứng tối thiểu: 8 GB RAM (CICFlowMeter Java heap + PyTorch), CPU Intel Core i5 Gen 8 trở lên. GPU không bắt buộc cho inference — CPU inference đủ với batch nhỏ (dưới 1 giây mỗi batch 128 flows).

B. THÔNG SỐ MÔ HÌNH VÀ KẾT QUẢ CHI TIẾT

B.1 Siêu tham số mô hình

B.1.1 NSL-KDD — Stage 1: Autoencoder Anomaly Gate

Bảng B.1: Siêu tham số Autoencoder NSL-KDD Stage 1

Tham số	Giá trị
Kiến trúc Encoder	$122 \rightarrow 64 \rightarrow 32 \rightarrow 16$
Kiến trúc Decoder	$16 \rightarrow 32 \rightarrow 64 \rightarrow 122$
Activation	ReLU
Optimizer	Adam (lr=0,001)
Epochs	100 (Early Stopping patience=10)
Batch size	256
Reconstruction loss	MSE
Ngưỡng τ	0,008481 (percentile 95 của Normal train errors)
Huấn luyện trên	Normal flows only (KDDTrain+)

B.1.2 NSL-KDD — Stage 2: FT-Transformer

Bảng B.2: Siêu tham số FT-Transformer NSL-KDD Stage 2

Tham số	Giá trị
d_{model}	128
n_{heads}	8
n_{layers}	4
d_{ff} (FFN)	512
Dropout	0,1
Activation	GELU
Normalization	Pre-LN
Loss	Focal Loss ($\gamma = 2$)
Optimizer	AdamW (lr=1e-4, weight_decay=1e-5)
Scheduler	CosineAnnealingLR
Epochs	50 (Early Stopping patience=5)
Batch size	256
Số features đầu vào	122 (sau one-hot encoding)
Số lớp đầu ra	5 (Normal, DoS, Probe, R2L, U2R)

B.1.3 NSL-KDD — Stage 2: LightGBM

Bảng B.3: Siêu tham số LightGBM NSL-KDD Stage 2

Tham số	Giá trị
n_estimators	1000
max_leaves	127
learning_rate	0,05
min_child_samples	20
feature_fraction	0,8
bagging_fraction	0,8
bagging_freq	5
class_weight	inverse frequency
early_stopping_rounds	50

B.1.4 NSL-KDD — Stage 2: Meta Logistic Regression

Bảng B.4: Siêu tham số Meta Logistic Regression (Stacking)

Tham số	Giá trị
Input dimension	10 (5 xác suất $\times$ 2 base learners)
C (regularization)	1,0
max_iter	1000
solver	lbfgs
multi_class	multinomial

B.1.5 CIC-IDS-2017 — Stage 1: Gating Network

Bảng B.5: Siêu tham số Gating Network FT-Transformer (CIC Stage 1)

Tham số	Giá trị
d_{model}	128
n_{heads}	8
n_{layers}	4
d_{ff} (FFN)	512
Dropout	0,2
Loss	CB-Focal Loss ($\gamma = 2$, α nghịch effective number)
Optimizer	AdamW (lr=1e-4)
Epochs	30 (Early Stopping patience=5)
Batch size	512
Số features đầu vào	77
Số lớp đầu ra	6 (Benign, DoS, DDoS, PortScan, Brute Force, Suspicious)
Ngưỡng phân loại	0,85 ($P(\text{Suspicious}) \geq 85\% \rightarrow$ chuyển Stage 2; ngược lại Benign)

B.1.6 CIC-IDS-2017 — Stage 2: Expert Network

Bảng B.6: Siêu tham số Expert Network FT-Transformer (CIC Stage 2)

Tham số	Giá trị
d_{model}	64
n_{heads}	4
n_{layers}	3
d_{ff} (FFN)	256
Dropout	0,2
Loss	CB-Focal Loss ($\gamma = 1,5$) + HNM ($w_{hard} = 2,0$, 2 vòng)
Optimizer	AdamW (lr=5e-5)
Epochs	50 (Early Stopping patience=7)
Batch size	256
Số features đầu vào	34 (sau lọc correlation > 0,95)
Số lớp đầu ra	5 (Benign, Web Attack, Bot, Infiltration, Heartbleed)

B.1.7 CIC-IDS-2017 — Stage 2: Random Forest và KNN

Bảng B.7: Siêu tham số Random Forest và KNN (Ensemble)

Tham số	Random Forest	KNN
n_estimators / K	150	16
max_depth / weights	25	uniform
max_features / metric	20	euclidean
min_samples_split / algorithm	5	ball_tree
class_weight / leaf_size	balanced	30

B.1.8 Testbed V8.5 — FT-Transformer

Bảng B.8: Siêu tham số FT-Transformer Testbed V8.5

Tham số	Giá trị
d_{model}	128
n_{heads}	8
n_{layers}	4
d_{ff} (FFN)	512
Dropout	0,15
Loss	CB-Focal Loss ($\gamma = 2$)
Optimizer	AdamW (lr theo lớp: backbone 1e-5, mid 1,5e-5, head 3e-5)
Weight decay	2e-4
Label smoothing	0,10
Epochs	50 (Early Stopping patience=5, dừng tại epoch 6)
Batch size	256
Số features đầu vào	80 (77 đặc trưng pipeline CIC + 3 đặc trưng Model Surgery)
Số lớp đầu ra	5 (Benign, BruteForce, DoS, PortScan, WebAttack)

3 đặc trưng bổ sung cho môi trường NAT:

- $IAT_CV = \text{std}(IAT) / \text{mean}(IAT)$ — hệ số biến thiên inter-arrival time
- $Bwd_Pkt_Ratio = Bwd_Packets / Total_Packets$ — tỷ lệ gói tin chiều ngược
- $Pkt_Size_Ratio = Bwd_Pkt_Len_Mean / Fwd_Pkt_Len_Mean$ — tỷ lệ kích thước gói tin

B.2 Kết quả chi tiết các thực nghiệm

B.2.1 NSL-KDD — Kết quả per-class trên KDDTest+

Bảng B.9: Kết quả per-class Stacking Ensemble trên KDDTest+

Lớp	Precision	Recall	F1	Support
Normal	0,7154	0,9682	0,8228	9.711
DoS	0,9630	0,8032	0,8759	7.458
Probe	0,8024	0,7852	0,7937	2.421
R2L	0,9681	0,2523	0,4003	2.885
U2R	0,5517	0,4776	0,5120	67
Macro F1	0,6809			

B.2.2 CIC-IDS-2017 — Kết quả ablation kiến trúc

Bảng B.10: Ablation study các cải tiến kiến trúc trên CIC-IDS-2017

Kiến trúc	Accuracy	Macro F1
1-Stage 9-class FTT	98,21%	0,7831
Two-Stage (không HNM)	99,55%	0,8817
+ HNM vòng 1	99,58%	0,9102
+ HNM vòng 2	99,60%	0,9181
+ HNM + Majority Vote (2/3)	99,61%	0,9247
+ HNM + Asymmetric Vote	99,55%	0,9294

B.2.3 CIC-IDS-2017 — Kết quả per-class cuối

Bảng B.11: Kết quả per-class Asymmetric Ensemble Voting trên CIC-IDS-2017

Lớp	Precision	Recall	F1	Support
Benign	0,9992	0,9953	0,9972	2.031.715
DoS	0,9683	0,9963	0,9821	225.024
DDoS	0,9984	0,9982	0,9983	114.453
PortScan	0,9936	0,9990	0,9963	142.079
BruteForce	0,9473	0,9989	0,9724	12.369
Web Attack	0,9084	0,9810	0,9433	1.950
Botnet	0,7947	0,6826	0,7344	1.758
Infiltration	0,9524	0,6061	0,7407	33
Heartbleed	1,0000	1,0000	1,0000	10
Macro F1			0,9294	

B.2.4 Chẩn đoán covariate shift

Bảng B.12: Kết quả chuỗi can thiệp domain adaptation

Can thiệp	Accuracy	MCC	CF (%)
Mô hình CIC trên CIC test	99,55%	0,9941	—
Direct transfer → Testbed WSL2	21,93%	−0,015	—
Re-fit Scaler đơn thuần	6,87%	+0,006	—
Layer Freezing Fine-tuning	99,82%	0,6825	0,61%
+ Model Surgery (77→80 features)	99,87%	0,7333	0,61%

B.2.5 Testbed V8.5 — Kết quả per-class trên val set đa dạng miền

Bảng B.13: Kết quả per-class mô hình V8.5 trên val set đa dạng miền

Lớp	Precision	Recall	F1	Support	Nguồn val
Benign	0,87	0,95	0,91	5.200	Testbed nội bộ
BruteForce	0,83	0,90	0,86	798	hydra + CIC Patator
DoS	0,98	0,89	0,93	1.978	Testbed nội bộ
PortScan	1,00	1,00	1,00	500	CIC Friday (surrogate)
Web Attack	0,96	0,81	0,88	2.707	Testbed nội bộ
Macro avg	0,93	0,91	0,917	11.183	
Balanced Accuracy			91,1%		
MCC			0,865		

B.2.6 So sánh val set đồng nhất và đa dạng miền

Bảng B.14: Ảnh hưởng của tính đồng nhất miền trong tập kiểm định lên BruteForce F1

Tập kiểm định	BruteForce F1	DoS F1	Macro F1
V8.4 — 100% CIC Patator (đồng nhất)	1,000	0,95	0,970
V8.5 — Mixed hydra + CIC Patator	0,860	0,93	0,916
Chênh lệch	−0,140	−0,02	−0,054